📋 MEDFLOW — DOCUMENTAÇÃO TÉCNICA COMPLETA E DETALHADA

Sistema de Gestão de Clínica Médica

Documento Privado — Contém credenciais, rotas internas e toda a lógica do sistema.
Versão: 2.9 | Data: 29 de Setembro de 2026
Criado para pessoas que **nunca tiveram contato com programação**. Cada conceito é explicado do zero.

➕ SOBRE ESTA VERSÃO (2.9 revisada, com adições)

O texto original foi mantido **integralmente, sem remover nem alterar nada**. Tudo o que foi acrescentado aparece em **caixas com fundo âmbar** como esta, ou nas **PARTES 13, 14 e 15** (cabeçalho laranja) no final. Também foram adicionados: **Sumário** com números de página, **9 figuras** (comparações e diagramas), **exemplos práticos** (requisições, respostas, código ilustrativo, cenários) e **exercícios**. A Parte 15 traz uma **avaliação de até onde um iniciante sem nenhum conhecimento prévio chega** lendo este documento.

Onde uma explicação nova foi *deduzida* a partir das rotas, permissões e DTOs (e não de uma leitura do código-fonte), isso está indicado com a palavra **"inferido"** ou **"confirmar no código"**.

Sumário

PARTE 1 — VISÃO GERAL: O QUE É O MEDFLOW? 4
1.1. O que este sistema faz? 4
1.2. Como o sistema está organizado? 4

PARTE 2 — TECNOLOGIAS: O QUE É CADA PROGRAMA E PRA QUE SERVE 7
2.1. Linguagens de Programação 7
2.2. Frameworks (Ferramentas Prontas) 7
2.3. Banco de Dados 9
2.4. Mensageria 9
2.5. Monitoramento 10
2.6. Containerização e Orquestração 11
2.7. Outras Ferramentas 11

PARTE 3 — ESTRUTURA DE PASTAS: O QUE É CADA PASTA E POR QUE ESTÁ ASSIM .. 14

PARTE 4 — ARQUITETURA DO BACKEND: CAMADA POR CAMADA 18
4.1. Camada de Entidades (entity/) 18
4.2. Camada de DTOs (dto/) 23
4.3. Camada de Repositórios (repository/) 26
4.4. Camada de Serviços (service/) — LÓGICA DE NEGÓCIO 28
4.5. Camada de Controllers (controller/) — ROTAS HTTP 33

PARTE 5 — SEGURANÇA: COMO FUNCIONA A PROTEÇÃO 39
5.1. Fluxo de Autenticação (Login) 39
5.2. Fluxo de Autorização (Acessar recurso protegido) 39
5.3. Tabela de Permissões por Role 39
5.4. Senhas e Criptografia 40

PARTE 6 — BANCO DE DADOS: ESTRUTURA DAS TABELAS 43
6.1. Diagrama de Relacionamento 43
6.2. Migrações SQL (na ordem de execução) 43

PARTE 7 — FRONTEND: COMO FUNCIONA A INTERFACE 45
7.1. Tecnologias do Frontend 45
7.2. Páginas 45
7.3. Componentes de Infraestrutura 46
7.4. Build e Deploy 46

PARTE 8 — INFRAESTRUTURA: DOCKER, KUBERNETES E MONITORAMENTO 48
8.1. Docker Compose (docker-compose.yml) 48
8.2. Kubernetes (k8s/) 49
8.3. Prometheus (prometheus/prometheus.yml) 49

PARTE 9 — FLUXOS COMPLETOS DO SISTEMA 51
9.1. Fluxo Completo: Paciente faz consulta do início ao fim 51

PARTE 10 — CREDENCIAIS E DADOS DE ACESSO 54
10.1. Banco de Dados PostgreSQL 54
10.2. RabbitMQ 54
10.3. Grafana 54
10.4. Prometheus 54
10.5. Swagger (Documentação da API) 54
10.6. Frontend (Interface Web) 54
10.7. Backend (API REST) 54
10.8. Usuário Admin Padrão 55

10.9. Chave Secreta JWT 55

PARTE 11 — COMO RODAR O SISTEMA 56

11.1. Pré-requisitos 56
11.2. Passo a passo 56
11.3. Parar o sistema 56
11.4. Parar e APAGAR todos os dados 56

PARTE 12 — GLOSSÁRIO DE TERMOS TÉCNICOS 58

PARTE 13 — RECOMENDAÇÕES DE SEGURANÇA PARA PRODUÇÃO 60

13.1. Credenciais padrão que precisam ser trocadas 60
13.2. Reduzir o que fica exposto 60
13.3. Token, sessão e navegador 60
13.4. Comunicação e infraestrutura 60
13.5. Dados de saúde e LGPD 61

PARTE 14 — PONTOS DE ATENÇÃO ENCONTRADOS NA REVISÃO 62

PARTE 15 — AVALIAÇÃO: ATÉ ONDE UM INICIANTE CHEGA COM ESTE DOCUMENTO 63

15.1. Veredito 63
15.2. A escala usada 63
15.3. O que funciona muito bem para um iniciante 64
15.4. O que ainda atrapalha quem parte do zero 64
15.5. Ordem de leitura sugerida para iniciantes 65
15.6. Exercícios práticos (com o resultado esperado) 65
15.7. O que estudar para subir do nível 3 para o 4 e 5 66
15.8. Conclusão 66

PARTE 1 — VISÃO GERAL: O QUE É O MEDFLOW?

1.1. O que este sistema faz?

O MedFlow é um **sistema completo de gestão de clínica médica** construído do zero. Ele funciona via internet (como um site) e gerencia todo o ciclo de vida de uma consulta médica: desde o momento em que o paciente se cadastra, passa pela recepção, faz triagem, é atendido pelo médico, recebe receitas e atestados, até o pagamento e faturamento final. Além disso, o sistema registra um rastro de auditoria de tudo que acontece.

Imagine uma clínica real. Nela existem diferentes pessoas com diferentes funções:

- O **Paciente** que vem para ser atendido
- A **Recepcionista** que agenda e faz check-in
- O **Enfermeiro de Triagem** que mede pressão, temperatura, etc.
- O **Médico** que atende, prescreve e faz diagnóstico
- O **Setor de Faturamento** que cuida dos pagamentos
- O **Administrador** que gerencia tudo

O MedFlow é o software que conecta todas essas pessoas em um único lugar, com controle de acesso (cada pessoa só vê o que pode ver) e rastreabilidade total.

1.2. Como o sistema está organizado?

O MedFlow usa uma arquitetura chamada **"Cliente-Servidor"** com microserviços containerizados. Vou explicar cada termo:

O que é "Cliente-Servidor"?

Pense em um restaurante. O **cliente** (frontend) é o cardápio bonito que fica na mesa — é o que o usuário vê e interage. O **servidor** (backend) é a cozinha — é onde toda a lógica e o trabalho pesado acontecem. O garçom (a internet/rede) leva os pedidos do cardápio para a cozinha e traz a comida de volta.

O que são "Containers" (Docker)?

Imagine que você quer montar uma loja. Em vez de construir do zero, você usa **containers de transporte** (aqueles de navio) que já vêm prontos com tudo dentro. Docker faz exatamente isso com software: ele empacota cada parte do sistema em uma "caixa" isolada que funciona igualzinho em qualquer computador. Isso elimina o problema de "na minha máquina funciona".

As "caixas" (containers) do MedFlow:

1. **Backend** (a cozinha) — Aplicação Java/Spring Boot
2. **Frontend** (o cardápio) — Aplicação React/TypeScript
3. **Banco de Dados** (o estoque) — PostgreSQL
4. **Mensageria** (o interfone entre setores) — RabbitMQ

5. **Monitoramento** (câmeras de segurança) — Prometheus + Grafana

> **➕ ADIÇÃO — Quantos containers, afinal? 5 ou 6?**
>
> A lista acima agrupa o sistema por **função** (5 grupos). No `docker-compose.yml` (Parte 8) existem **6 containers**, porque o grupo "Monitoramento" é formado por dois: **Prometheus** e **Grafana**. Os 6 são: `db`, `rabbitmq`, `backend`, `frontend`, `prometheus` e `grafana`. A tabela de portas está na seção 11.6.

> **➕ ADIÇÃO — Comparação: o MedFlow é como um restaurante**
>
> A analogia da seção 1.2 pode ser levada mais longe. A figura abaixo coloca lado a lado cada peça de um restaurante e a peça equivalente no MedFlow.

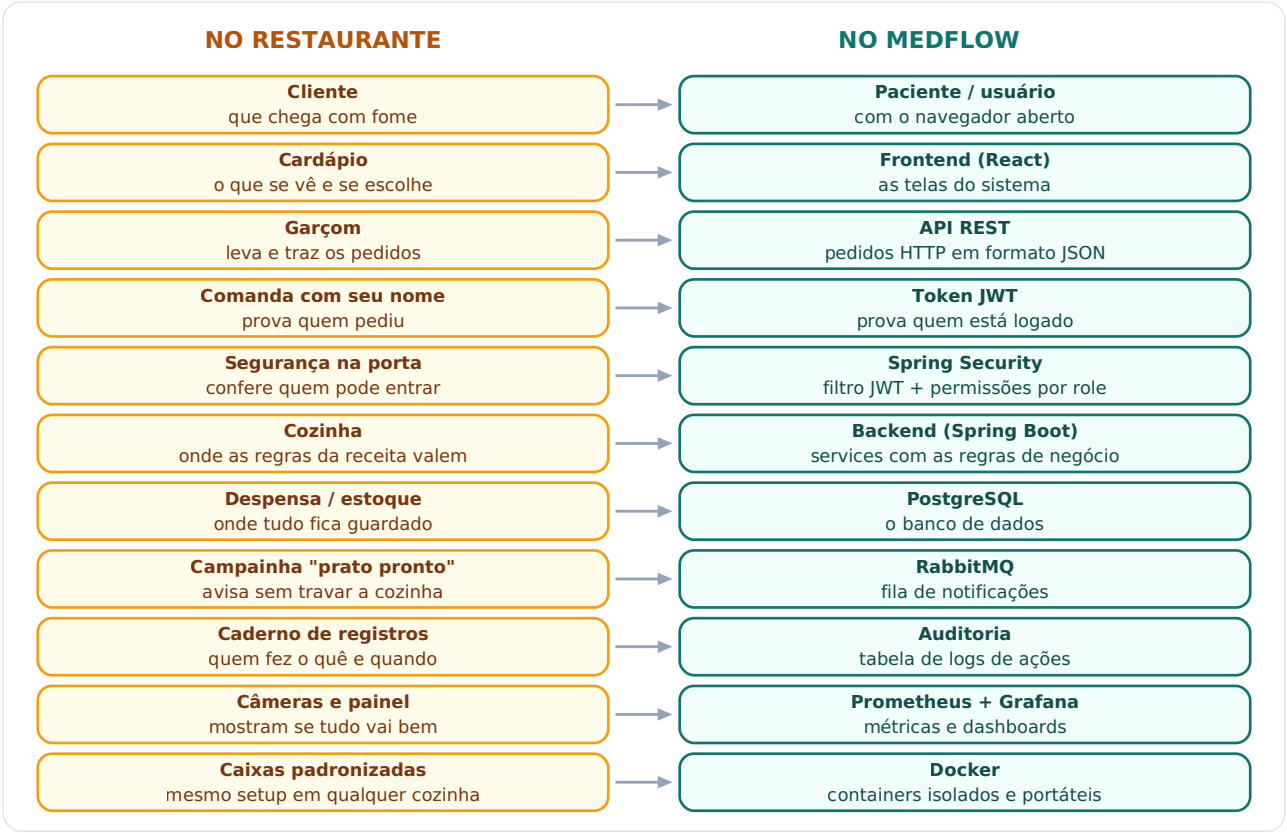


Figura 1 — Comparação entre um restaurante e o MedFlow

➕ ADIÇÃO — Mapa geral: quem conversa com quem

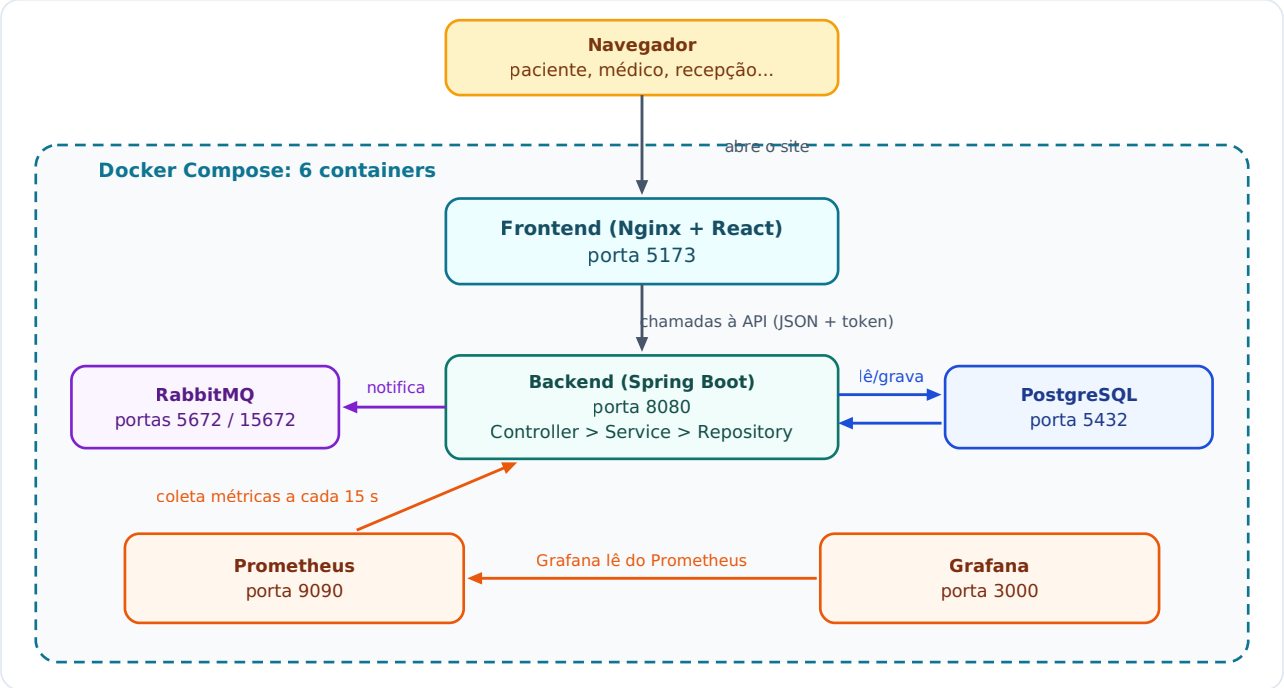


Figura 2 — Arquitetura do MedFlow: os 6 containers, as portas e quem se comunica com quem

Leitura rápida da figura: o navegador só fala com o **frontend**; o frontend só fala com o **backend**; somente o backend fala com o **banco** e o **RabbitMQ**; o **Prometheus** busca as métricas do backend e o **Grafana** desenha gráficos com o que o Prometheus guardou.

PARTE 2 — TECNOLOGIAS: O QUE É CADA PROGRAMA E PRA QUE SERVE

2.1. Linguagens de Programação

Java 21

- **O que é?** Uma das linguagens de programação mais usadas no mundo, especialmente para sistemas empresariais e bancários. Foi criada pela Sun Microsystems em 1995 e hoje pertence à Oracle.
- **Pra que serve aqui?** É a linguagem do backend (servidor). Todo o código que processa dados, valida regras de negócio, se comunica com o banco de dados e gera as respostas é escrito em Java.
- **Por que Java?** É robusta (poucos erros), segura (muito usada em bancos), e tem uma comunidade gigante de desenvolvedores.
- **Versão 21:** É a versão mais recente com suporte de longo prazo (LTS), o que significa que vai receber atualizações de segurança por anos.

TypeScript

- **O que é?** Uma versão "melhorada" do JavaScript (a linguagem que todo site usa). O TypeScript adiciona "tipos" — ou seja, você define que uma variável é um número, um texto, etc., e o sistema avisa se você usar errado.
- **Pra que serve aqui?** É a linguagem do frontend (interface do usuário). Tudo que o usuário vê — botões, formulários, tabelas — é feito com TypeScript.

SQL (Structured Query Language)

- **O que é?** A linguagem para conversar com bancos de dados. É como falar com o estoquista: "Me dá todos os pacientes que têm consulta hoje" ou "Cadastra esse novo médico".
- **Pra que serve aqui?** Criar as tabelas do banco, fazer consultas, inserir dados.

HTML e CSS

- **O que é?** HTML é a linguagem que define a estrutura de uma página web (títulos, parágrafos, botões). CSS é o que define a aparência (cores, tamanhos, posições).
- **Pra que serve aqui?** São a base visual do frontend.

2.2. Frameworks (Ferramentas Prontas)

Spring Boot 4.1.1

- **O que é?** Um framework (kit de ferramentas pronto) para Java. Em vez de escrever tudo do zero, o Spring Boot já traz pronto: servidor web, conexão com banco de dados, segurança, etc. É como comprar uma casa pré-fabricada em vez de construir tijolo por tijolo.

- **Pra que serve aqui?** É o coração do backend. Ele:
  - Recebe as requisições HTTP (pedidos do frontend)
  - Processa as regras de negócio
  - Se comunica com o banco de dados
  - Retorna as respostas em JSON

Spring Security

- **O que é?** O módulo de segurança do Spring. Ele cuida de tudo relacionado a autenticação ("quem é você?") e autorização ("o que você pode fazer?").
- **Pra que serve aqui?** Garante que:
  - Ninguém acessa o sistema sem fazer login
  - Um paciente não vê dados de outro paciente
  - Só médicos podem criar prontuários
  - Só admins podem ver a auditoria

Spring Data JPA (Java Persistence API)

- **O que é?** Um módulo que facilita enormemente a comunicação com o banco de dados. Em vez de escrever SQL na mão, você simplesmente define objetos Java e o Spring gera o SQL automaticamente.
- **Pra que serve aqui?** Converter entre objetos Java (Paciente, Médico, Consulta) e as tabelas do banco de dados PostgreSQL.

React 18

- **O que é?** Uma biblioteca JavaScript criada pelo Facebook para construir interfaces de usuário. É usada por Instagram, WhatsApp Web, Netflix, Airbnb e milhares de outros sites.
- **Pra que serve aqui?** Construir todas as telas que o usuário vê: login, dashboard, lista de médicos, consultas, etc.

Tailwind CSS

- **O que é?** Um framework de CSS que oferece classes prontas para estilizar rapidamente. Em vez de escrever `color: blue; font-size: 16px;`, você escreve `text-blue-500 text-base` direto no HTML.
- **Pra que serve aqui?** Deixar o frontend bonito de forma rápida e consistente.

Vite

- **O que é?** Uma ferramenta moderna para desenvolvimento frontend. Ela faz o "build" (compilação) do código TypeScript/React para algo que o navegador entende, e fornece um servidor de desenvolvimento com atualização instantânea.
- **Pra que serve aqui?** Compilar e servir o frontend durante o desenvolvimento.

2.3. Banco de Dados

PostgreSQL 16

- **O que é?** Um dos bancos de dados relacionais mais poderosos e confiáveis do mundo. É gratuito e open-source. Grandes empresas como Apple, Spotify e Instagram usam PostgreSQL.
- **Pra que serve aqui?** Guardar TODOS os dados do sistema de forma permanente:
  - Cadastro de usuários, pacientes, médicos
  - Consultas agendadas
  - Prontuários médicos
  - Receitas e atestados
  - Pagamentos
  - Logs de auditoria
- **Por que PostgreSQL?** É extremamente confiável, suporta milhões de registros, tem recursos avançados (JSON, full-text search, UUIDs nativos) e é gratuito.
- **Credenciais configuradas:**
  - Usuário: `medflow`
  - Senha: `medflow`
  - Banco: `medflow`
  - Porta: `5432`

Flyway

- **O que é?** Uma ferramenta de "migração" de banco de dados. Ela garante que a estrutura do banco (tabelas, colunas) evolua de forma controlada e rastreável, como um controle de versão para o banco.
- **Pra que serve aqui?** Cada vez que o sistema sobe, o Flyway verifica se existem novas migrações (arquivos SQL numerados) que ainda não foram executadas. Se existem, ele executa na ordem. Isso garante que o banco está sempre atualizado, sem precisar recriar nada manualmente.
- **Migrações do MedFlow (na ordem de execução):**
  - `V1` — Criar tabelas iniciais (usuario, paciente, medico, consulta, etc.)
  - `V2` — Criar tabela de funcionários (recepcionista, faturamento)
  - `V3` — Criar tabela de especialidades médicas (M:N)
  - `V4` — Remover coluna antiga de especialidade do médico
  - `V5` — Criar tabela de triagem
  - `V6` — Criar tabelas de receitas, atestados, e evoluir prontuário
  - `V7` — Criar tabela de auditoria

2.4. Mensageria

RabbitMQ

- **O que é?** Um sistema de filas de mensagens. Imagine uma caixa de correio: em vez de entregar uma carta na mão da pessoa (que pode não estar em casa), você coloca na caixa e ela pega quando puder. RabbitMQ faz isso com mensagens entre diferentes partes do sistema.

- **Pra que serve aqui?** Quando uma consulta é agendada, remarcada, cancelada ou confirmada, o sistema coloca uma mensagem na fila `notificacoes`. Um "ouvinte" (listener) no backend consome essas mensagens e pode enviar e-mails, SMS, push notifications, etc. Hoje ele só imprime no console, mas a infraestrutura está pronta para qualquer canal de notificação.
- **Credenciais:**
  - Usuário: `guest`
  - Senha: `guest`
  - Porta AMQP: `5672`
  - Porta do Painel Web: `15672` (acesse http://localhost:15672 para ver as filas)

2.5. Monitoramento

Prometheus

- **O que é?** Um sistema de monitoramento de código aberto criado pelo SoundCloud. Ele coleta métricas numéricas do sistema ao longo do tempo: quantas requisições por segundo, quanto de memória está usando, tempo de resposta, etc.
- **Pra que serve aqui?** Ele faz "scraping" (coleta) das métricas que o Spring Boot Actuator expõe no endpoint `/actuator/prometheus`. Isso inclui:
  - Número de requisições HTTP por endpoint
  - Tempo de resposta de cada requisição
  - Uso de memória da JVM (máquina virtual Java)
  - Status do banco de dados
  - Filas do RabbitMQ
- **Porta:** `9090` (acesse http://localhost:9090 para ver as métricas)

Grafana

- **O que é?** Uma plataforma de visualização de dados. Enquanto o Prometheus coleta os números, o Grafana transforma esses números em gráficos bonitos e dashboards interativos.
- **Pra que serve aqui?** Criar dashboards visuais para acompanhar a saúde do sistema em tempo real:
  - Gráficos de uso de CPU e memória
  - Taxa de erros vs. taxa de sucesso
  - Tempo médio de resposta das APIs
  - Alertas quando algo está errado
- **Credenciais:**
  - Usuário: `admin`
  - Senha: `admin`
  - Porta: `3000` (acesse http://localhost:3000)

2.6. Containerização e Orquestração

Docker

- **O que é?** Uma plataforma que permite empacotar aplicações em "containers" — ambientes isolados e leves que contêm tudo que a aplicação precisa para rodar (código, bibliotecas, configurações).
- **Pra que serve aqui?** Cada componente do MedFlow (backend, frontend, banco, rabbitmq, prometheus, grafana) roda em seu próprio container Docker. Isso garante:
  - Isolamento: um componente não interfere no outro
  - Portabilidade: funciona igualzinho em qualquer máquina
  - Reprodutibilidade: qualquer pessoa pode rodar o sistema com um único comando

Docker Compose

- **O que é?** Uma ferramenta que permite definir e rodar múltiplos containers Docker de uma vez só, usando um arquivo de configuração (`docker-compose.yml`).
- **Pra que serve aqui?** Com um único comando (`docker compose up`), todos os 6 containers sobem juntos, na ordem correta, com todas as configurações. Sem ele, teríamos que subir cada container manualmente.

Kubernetes (K8s)

- **O que é?** Um sistema de orquestração de containers criado pelo Google. Se o Docker Compose é como ter 6 jarros de plantas na janela, o Kubernetes é como ter uma estufa automatizada com irrigação, controle de temperatura e replantio automático.
- **Pra que serve aqui?** O MedFlow tem arquivos de configuração Kubernetes (na pasta `k8s/`) preparados para deploy em produção. O Kubernetes oferece:
  - **Escalabilidade automática:** Se muitos pacientes acessam ao mesmo tempo, ele cria mais cópias do backend automaticamente
  - **Auto-recuperação:** Se um container morre, o Kubernetes reinicia sozinho
  - **Load balancing:** Distribui o tráfego entre as cópias
  - **Rolling updates:** Atualiza o sistema sem downtime (sem ficar fora do ar)
- **Arquivos K8s do MedFlow:**
  - `k8s/backend.yaml` — Deploy do backend (2 réplicas)
  - `k8s/frontend.yaml` — Deploy do frontend (2 réplicas)
  - `k8s/postgres.yaml` — Deploy do banco de dados (com PersistentVolumeClaim para não perder dados)
  - `k8s/rabbitmq.yaml` — Deploy do RabbitMQ

2.7. Outras Ferramentas

Maven

- **O que é?** Um gerenciador de dependências e build para Java. Pense nele como um gerente de compras: você lista tudo que precisa (bibliotecas) em um arquivo (`pom.xml`) e ele baixa, organiza e compila tudo para você.

- **Pra que serve aqui?** Gerenciar todas as bibliotecas do backend (Spring, JWT, PostgreSQL driver, etc.) e compilar o código Java em um arquivo executável (.jar).

Lombok

- **O que é?** Uma biblioteca Java que elimina código repetitivo ("boilerplate"). Em vez de escrever 50 linhas de getters, setters e construtores, você coloca uma anotação `@Data` e o Lombok gera tudo automaticamente.
- **Pra que serve aqui?** Todas as entidades e DTOs usam `@Data` (gera getters/setters), `@RequiredArgsConstructor` (gera construtor com dependências).

JWT (JSON Web Token)

- **O que é?** Um padrão aberto para criar tokens de autenticação seguros. Funciona como um crachá digital: quando você faz login, o sistema gera um token (uma string longa e criptografada) que contém seus dados (email, role). Esse token é enviado em todas as próximas requisições para provar sua identidade.
- **Pra que serve aqui?** Autenticação do sistema. Após o login, o frontend guarda o token JWT no localStorage e o envia em cada requisição no header `Authorization: Bearer <token>`.

Swagger / OpenAPI (SpringDoc)

- **O que é?** Uma ferramenta que gera documentação interativa da API automaticamente. Ela lê o código e cria uma página web onde você pode ver todas as rotas, parâmetros e até testar as requisições direto no navegador.
- **Pra que serve aqui?** Acesse http://localhost:8080/swagger-ui.html para ver e testar todas as APIs do MedFlow sem precisar de nenhuma ferramenta adicional.

Micrometer

- **O que é?** Uma biblioteca de instrumentação para Java que coleta métricas da aplicação e as expõe em formatos que o Prometheus entende.
- **Pra que serve aqui?** Expõe métricas do Spring Boot (requisições, erros, tempo de resposta, memória) no endpoint `/actuator/prometheus`.

Nginx

- **O que é?** Um servidor web de altíssima performance. É usado por metade dos sites mais visitados do mundo.
- **Pra que serve aqui?** No container do frontend, o Nginx serve os arquivos estáticos (HTML, CSS, JS) do React e faz proxy reverso para o backend, redirecionando chamadas de `/api/*` para o servidor Java.

Git

- **O que é?** Um sistema de controle de versão. Ele rastreia todas as mudanças feitas no código ao longo do tempo, permitindo voltar para versões anteriores, trabalhar em equipe, e manter um histórico completo.
- **Pra que serve aqui?** Todo o código do MedFlow está versionado no GitHub: https://github.com/Samuka310/medflow

➕ ADIÇÃO — Java 21 e "LTS"

LTS (*Long Term Support*) é uma versão que recebe correções e atualizações de segurança por muitos anos. O texto da seção 2.1 diz que o Java 21 é a LTS "mais recente". Isso era verdade quando ele foi escolhido; hoje (setembro de 2026) o **Java 25**, lançado em setembro de 2025, também é LTS. O Java 21 continua válido e suportado; migrar para o 25 é uma evolução futura opcional, sem urgência.

➕ ADIÇÃO — O que é o Spring Boot Actuator

Citado nas seções do Prometheus e do Micrometer sem definição. É um módulo do Spring Boot que **expõe endpoints de monitoramento** da aplicação, como `/actuator/health` (a aplicação está saudável?) e `/actuator/prometheus` (métricas em formato que o Prometheus lê).

➕ ADIÇÃO — Mapa das tecnologias por camada

Para não se perder nas dezenas de nomes desta parte, use a tabela abaixo: cada ferramenta aparece em **uma** linha, agrupada pelo problema que resolve.

Camada	Ferramentas	Em uma frase
Telas (Frontend)	React, TypeScript, Vite, Tailwind CSS, Axios, React Router	Desenham as páginas e conversam com o backend
Regras (Backend)	Java 21, Spring Boot, Spring Data JPA, Lombok, Maven	Recebem pedidos, aplicam as regras e acessam o banco
Segurança	Spring Security, JWT, BCrypt	Garantem quem é você e o que você pode fazer
Dados	PostgreSQL, Flyway	Guardam as informações e versionam a estrutura das tabelas
Mensagens	RabbitMQ	Entregam avisos de forma assíncrona
Documentação	Swagger / OpenAPI	Mostram e permitem testar todas as rotas
Empacotamento e execução	Docker, Docker Compose, Kubernetes, Nginx	Fazem o sistema rodar igual em qualquer lugar
Observação	Micrometer, Actuator, Prometheus, Grafana	Medem e mostram a saúde do sistema
Histórico do código	Git / GitHub	Guardam todas as versões do projeto

PARTE 3 — ESTRUTURA DE PASTAS: O QUE É CADA PASTA E POR QUE ESTÁ ASSIM

```
Medflow/                          ← RAIZ DO PROJETO
│
├── docker-compose.yml             ← Arquivo que orquestra TODOS os containers
│
├── backend/                       ← SERVIDOR (a "cozinha" do sistema)
│   ├── Dockerfile                 ← Receita para construir o container do backend
│   ├── pom.xml                    ← Lista de dependências Java (Maven)
│   └── src/
│       ├── main/
│       │   ├── java/com/medflow/
│       │   │   ├── MedflowApplication.java     ← Ponto de partida do sistema
│       │   │   ├── config/                      ← Configurações do sistema
│       │   │   │   ├── AppConfig.java           ← Config geral (PasswordEncoder)
│       │   │   │   ├── CorsConfig.java          ← Config de Cross-Origin (CORS)
│       │   │   │   ├── OpenApiConfig.java       ← Config do Swagger
│       │   │   │   ├── RabbitMQConfig.java      ← Config das filas RabbitMQ
│       │   │   │   └── SecurityConfig.java      ← Config do Spring Security
│       │   │   ├── security/                    ← Segurança (JWT)
│       │   │   │   ├── JwtService.java          ← Gera e valida tokens JWT
│       │   │   │   └── JwtAuthenticationFilter.java  ← Filtro que intercepta requisições
│       │   │   ├── entity/                      ← Entidades (representam tabelas do banco)
│       │   │   │   ├── Usuario.java
│       │   │   │   ├── Paciente.java
│       │   │   │   ├── Medico.java
│       │   │   │   ├── Funcionario.java
│       │   │   │   ├── Consulta.java
│       │   │   │   ├── Prontuario.java
│       │   │   │   ├── Pagamento.java
│       │   │   │   ├── Especialidade.java
│       │   │   │   ├── Exame.java
│       │   │   │   ├── Triagem.java
│       │   │   │   ├── Receita.java
│       │   │   │   ├── Atestado.java
│       │   │   │   └── Auditoria.java
│       │   │   ├── dto/                         ← Objetos de transferência de dados
│       │   │   │   ├── LoginRequest.java
│       │   │   │   ├── LoginResponse.java
│       │   │   │   ├── RegisterRequest.java
│       │   │   │   ├── ConsultaDTO.java
│       │   │   │   ├── PacienteDTO.java
│       │   │   │   ├── PacienteClinicoDTO.java
│       │   │   │   ├── MedicoDTO.java
│       │   │   │   ├── ProntuarioDTO.java
│       │   │   │   ├── PagamentoDTO.java
│       │   │   │   ├── ExameDTO.java
│       │   │   │   ├── EspecialidadeDTO.java
│       │   │   │   ├── TriagemDTO.java
│       │   │   │   ├── ReceitaDTO.java
│       │   │   │   ├── AtestadoDTO.java
│       │   │   │   ├── AuditoriaDTO.java
│       │   │   │   ├── FaturamentoRelatorioDTO.java
│       │   │   │   └── FuncionarioRequest.java
│       │   │   ├── repository/                  ← Interfaces de acesso ao banco
│       │   │   │   ├── UsuarioRepository.java
│       │   │   │   ├── PacienteRepository.java
│       │   │   │   ├── MedicoRepository.java
│       │   │   │   ├── ConsultaRepository.java
│       │   │   │   ├── ProntuarioRepository.java
│       │   │   │   ├── PagamentoRepository.java
│       │   │   │   ├── EspecialidadeRepository.java
│       │   │   │   ├── ExameRepository.java
│       │   │   │   ├── FuncionarioRepository.java
│       │   │   │   ├── TriagemRepository.java
│       │   │   │   ├── ReceitaRepository.java
│       │   │   │   ├── AtestadoRepository.java
```

```
│   │   │   │   │   └── AuditoriaRepository.java
│   │   │   │   ├── service/                    ← Lógica de negócio
│   │   │   │   │   ├── AuthService.java
│   │   │   │   │   ├── AdminService.java
│   │   │   │   │   ├── ConsultaService.java
│   │   │   │   │   ├── PacienteService.java
│   │   │   │   │   ├── MedicoService.java
│   │   │   │   │   ├── ProntuarioService.java
│   │   │   │   │   ├── PagamentoService.java
│   │   │   │   │   ├── EspecialidadeService.java
│   │   │   │   │   ├── ExameService.java
│   │   │   │   │   ├── TriagemService.java
│   │   │   │   │   ├── FaturamentoService.java
│   │   │   │   │   ├── AuditoriaService.java
│   │   │   │   │   └── NotificacaoListener.java
│   │   │   │   ├── controller/                 ← Portas de entrada (rotas HTTP)
│   │   │   │   │   ├── AuthController.java
│   │   │   │   │   ├── AdminController.java
│   │   │   │   │   ├── ConsultaController.java
│   │   │   │   │   ├── PacienteController.java
│   │   │   │   │   ├── MedicoController.java
│   │   │   │   │   ├── ProntuarioController.java
│   │   │   │   │   ├── PagamentoController.java
│   │   │   │   │   ├── EspecialidadeController.java
│   │   │   │   │   ├── ExameController.java
│   │   │   │   │   ├── FaturamentoController.java
│   │   │   │   │   └── AuditoriaController.java
│   │   │   │   └── exception/                  ← Tratamento de erros
│   │   │   │       ├── ErrorResponse.java
│   │   │   │       └── GlobalExceptionHandler.java
│   │   │   └── resources/
│   │   │       ├── application.yml              ← Config principal
│   │   │       ├── application-dev.yml           ← Config para desenvolvimento
│   │   │       ├── application-prod.yml          ← Config para produção
│   │   │       └── db/migration/                 ← Scripts de migração (Flyway)
│   │   │           ├── V1__criar_tabelas_iniciais.sql
│   │   │           ├── V2__criar_tabela_funcionario.sql
│   │   │           ├── V3__criar_tabela_medico_especialidade.sql
│   │   │           ├── V4__remover_coluna_especialidade_de_medico.sql
│   │   │           ├── V5__criar_tabela_triagem.sql
│   │   │           ├── V6__criar_receitas_atestados_evoluir_prontuario.sql
│   │   │           └── V7__criar_tabela_auditoria.sql
│   │   └── test/                                 ← Testes automatizados
│   │       └── java/com/medflow/
│   │           ├── MedflowApplicationTests.java
│   │           ├── SecurityIntegrationTest.java
│   │           └── service/
│   │               ├── AuthServiceTest.java
│   │               └── ConsultaServiceTest.java
│
├── frontend/                       ← INTERFACE DO USUÁRIO (o "cardápio")
│   ├── Dockerfile                  ← Receita para construir o container do frontend
│   ├── nginx.conf                  ← Configuração do servidor web Nginx
│   ├── package.json                ← Lista de dependências JavaScript
│   ├── vite.config.ts              ← Config do Vite (build tool)
│   ├── tsconfig.json               ← Config do TypeScript
│   ├── tailwind.config.js          ← Config do Tailwind CSS
│   ├── postcss.config.js           ← Config do PostCSS
│   ├── index.html                  ← Página HTML raiz
│   └── src/
│       ├── main.tsx                ← Ponto de entrada do React
│       ├── App.tsx                 ← Componente principal (rotas)
│       ├── App.css                 ← Estilos globais
│       ├── index.css               ← Estilos do Tailwind
│       ├── api/
│       │   └── axios.ts            ← Config do cliente HTTP (Axios)
│       ├── context/
│       │   └── AuthContext.tsx     ← Contexto de autenticação (estado global)
│       ├── components/
│       │   ├── Layout.tsx          ← Layout com menu lateral
│       │   └── ProtectedRoute.tsx ← Componente que protege rotas
│       └── pages/
│           ├── Login.tsx           ← Página de login
│           ├── Register.tsx        ← Página de cadastro
│           ├── Dashboard.tsx       ← Painel principal
```

```
│           ├── Medicos.tsx        ← Lista de médicos
│           └── MinhasConsultas.tsx ← Consultas do paciente
│
├── k8s/                          ← KUBERNETES (produção em nuvem)
│   ├── backend.yaml
│   ├── frontend.yaml
│   ├── postgres.yaml
│   └── rabbitmq.yaml
│
└── prometheus/                   ← MONITORAMENTO
    └── prometheus.yml            ← Config do Prometheus
```

Por que essa estrutura?

A organização segue o padrão **MVC (Model-View-Controller)** adaptado para Spring Boot:

- **Model (entity/):** Define como os dados são estruturados. Cada arquivo aqui representa uma tabela no banco de dados.
- **View (frontend/):** O que o usuário vê. Telas, botões, formulários.
- **Controller (controller/):** Recebe as requisições HTTP e delega para os serviços.
- **Service (service/):** Contém TODA a lógica de negócio. É onde as regras vivem.
- **Repository (repository/):** Interface de comunicação com o banco de dados.
- **DTO (dto/):** Objetos que definem exatamente quais dados entram e saem da API.
- **Config (config/):** Configurações do sistema (segurança, CORS, RabbitMQ).
- **Security (security/):** Tudo relacionado a autenticação JWT.
- **Exception (exception/):** Tratamento centralizado de erros.

➕ ADIÇÃO — Onde mexer quando eu quiser mudar algo?

Quero...	Onde mexer
Mudar uma regra (ex.: antecedência mínima para cancelar)	`service/ConsultaService.java`
Criar uma nova rota da API	`controller/` (com a permissão `@PreAuthorize`) + `service/`
Adicionar um campo em tabela existente	`entity/` + `dto/` + **nova migração** (ex.: `V8__...sql`)
Criar uma tabela nova	`entity/` + `repository/` + **nova migração**
Criar uma tela nova	`frontend/src/pages/` + rota em `App.tsx` + item de menu em `Layout.tsx`
Mudar portas, senhas do banco, variáveis	`docker-compose.yml` e `application*.yml`
Ajustar o monitoramento	`prometheus/prometheus.yml`

Padrão de nomes: para uma tabela `X` costumam existir `X.java` (entity), `XRepository`, `XService` (quando há regra de negócio), `XController` e `XDTO`. Saber isso ajuda a achar qualquer arquivo rapidamente.

Regra de ouro dos bancos de dados: nunca edite as migrações antigas (V1 a V7) depois de aplicadas. O Flyway guarda uma "assinatura" (checksum) de cada uma e se recusa a iniciar se ela mudar. Mudanças sempre entram como **uma migração nova**.

➕ ADIÇÃO — Exemplo: seguindo a "Consulta" por todas as pastas

Para entender a estrutura, escolha **um** conceito e veja onde ele aparece. Em cada pasta, o mesmo assunto ("consulta") tem um papel diferente:

Pasta	Arquivo	Papel
`db/migration/`	`V1__criar_tabelas_iniciais.sql`	Cria a tabela `consulta` no banco
`entity/`	`Consulta.java`	Representa uma linha da tabela em forma de objeto Java
`dto/`	`ConsultaDTO.java`	Define o formato dos dados que entram e saem pela API
`repository/`	`ConsultaRepository.java`	Busca e grava consultas no banco
`service/`	`ConsultaService.java`	Contém as regras (30 min, conflitos, ownership)
`controller/`	`ConsultaController.java`	Expõe as rotas `/api/consultas/...` e as permissões
`test/service/`	`ConsultaServiceTest.java`	Testa automaticamente as regras do serviço
`frontend/src/pages/`	`MinhasConsultas.tsx`	Mostra as consultas na tela do paciente

Quase todo assunto do sistema (Paciente, Pagamento, Prontuário...) segue **esse mesmo caminho**. Entendendo um, você entende todos.

PARTE 4 — ARQUITETURA DO BACKEND: CAMADA POR CAMADA

4.1. Camada de Entidades (entity/)

As entidades são classes Java que representam **exatamente** as tabelas do banco de dados. Cada campo da classe corresponde a uma coluna da tabela.

Usuario.java

É a entidade base de autenticação. Todo mundo que acessa o sistema (paciente, médico, recepcionista, admin) primeiro é um Usuário.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único universal (gerado automaticamente)
`nome`	String	Nome completo do usuário
`email`	String	E-mail (usado como login, deve ser único)
`senha`	String	Senha criptografada com BCrypt
`role`	String	Papel: ADMIN, PACIENTE, MEDICO, RECEPCIONISTA, TRIAGEM, FATURAMENTO

Implementa UserDetails do Spring Security, o que significa que o Spring sabe como usar esta classe para autenticação. O método `getAuthorities()` retorna a role com prefixo "ROLE_" (ex: `ROLE_ADMIN`), que é o padrão do Spring Security.

Paciente.java

Estende o conceito de Usuário para armazenar dados específicos de paciente.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único do paciente
`cpf`	String	CPF do paciente (único)
`dataNascimento`	LocalDate	Data de nascimento
`telefone`	String	Telefone de contato
`usuario`	Usuario	Referência ao usuário (ManyToOne)

Medico.java

Dados específicos de médico, incluindo CRM e valor da consulta.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`crm`	String	Registro no Conselho Regional de Medicina
`valorConsulta`	BigDecimal	Valor cobrado por consulta
`usuario`	Usuario	Referência ao usuário
`especialidades`	Set	Lista de especialidades (ManyToMany)

Funcionario.java

Para recepcionistas, técnicos de triagem e funcionários do faturamento.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`cargo`	String	RECEPCIONISTA, TRIAGEM ou FATURAMENTO
`dataAdmissao`	LocalDate	Data de admissão
`usuario`	Usuario	Referência ao usuário

Consulta.java

O coração do sistema. Cada consulta liga um paciente a um médico em uma data/hora.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`paciente`	Paciente	Quem vai ser atendido
`medico`	Medico	Quem vai atender
`dataHora`	LocalDateTime	Data e hora da consulta
`status`	String	AGENDADA, CONFIRMADA, AGUARDANDO_TRIAGEM, CANCELADA, REALIZADA

Fluxo de Status:

```
AGENDADA → CONFIRMADA → AGUARDANDO_TRIAGEM → REALIZADA
    ↓           ↓
CANCELADA    CANCELADA
```

Pagamento.java

Cada consulta gera automaticamente um pagamento pendente.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`consulta`	Consulta	A consulta associada (OneToOne)

Campo	Tipo	Descrição
`valor`	BigDecimal	Valor em reais
`status`	String	PENDENTE, PAGO, ESTORNADO
`formaPagamento`	String	CARTAO, PIX, DINHEIRO, etc.

Prontuario.java

Registro médico do atendimento.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`consulta`	Consulta	Consulta associada
`diagnosticoCid`	String	Código CID do diagnóstico
`evolucaoClinica`	String	Evolução clínica do paciente
`condutaMedica`	String	Conduta adotada pelo médico
`observacoes`	String	Observações gerais

Triagem.java

Dados de triagem (sinais vitais) coletados antes do atendimento médico.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`consulta`	Consulta	Consulta associada
`pressaoArterial`	String	Ex: "120/80"
`temperatura`	Double	Em graus Celsius
`frequenciaCardiaca`	Integer	Batimentos por minuto
`peso`	Double	Em kg
`altura`	Double	Em metros
`saturacao`	Integer	Saturação de oxigênio (%)
`queixaPrincipal`	String	O que o paciente está sentindo
`prioridade`	String	BAIXA, MEDIA, ALTA, EMERGENCIA

Especialidade.java

Especialidades médicas (Cardiologia, Dermatologia, etc.)

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único

Campo	Tipo	Descrição
`nome`	String	Nome da especialidade (único)

Exame.java

Exames associados a uma consulta.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`consulta`	Consulta	Consulta associada
`tipo`	String	Tipo do exame
`descricao`	String	Detalhes do exame
`resultado`	String	Resultado do exame
`status`	String	Status do exame

Receita.java

Prescrições médicas de medicamentos.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`paciente`	Paciente	Paciente que recebe
`medicamento`	String	Nome do medicamento
`dosagem`	String	Ex: "500mg"
`instrucoes`	String	Como tomar
`dataEmissao`	LocalDate	Data da emissão

Atestado.java

Atestados médicos emitidos pelo médico.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`paciente`	Paciente	Paciente
`descricao`	String	Motivo do atestado
`dias`	Integer	Quantidade de dias
`dataEmissao`	LocalDate	Data da emissão

Auditoria.java

Registro de todas as ações realizadas no sistema.

Campos:

Campo	Tipo	Descrição
`id`	UUID	Identificador único
`usuario`	Usuario	Quem fez a ação
`usuarioEmail`	String	E-mail de quem fez
`acao`	String	AGENDAR_CONSULTA, EVENTO_PAGAMENTO, etc.
`entidade`	String	Consulta, Pagamento, Prontuario
`entidadeId`	UUID	ID do objeto afetado
`detalhes`	String	Descrição da ação
`dataHora`	LocalDateTime	Quando aconteceu

➕ ADIÇÃO — Como ler os tipos e os relacionamentos das tabelas acima

Tipo	Significado	Exemplo no sistema
`UUID`	Código único de 36 caracteres, gerado automaticamente	`id` de todas as tabelas
`String`	Texto	nome, e-mail, status
`Integer`	Número inteiro	dias do atestado, batimentos
`Double`	Número decimal aproximado	temperatura, peso, altura
`BigDecimal`	Decimal **exato**, usado para dinheiro (Double faz arredondamentos imprecisos, ex.: 0,1 + 0,2)	`valorConsulta`, `valor` do pagamento
`LocalDate`	Só data (dia/mês/ano)	data de nascimento
`LocalDateTime`	Data e hora	horário da consulta
`Set<...>`	Conjunto de itens sem repetição	especialidades de um médico

Relacionamento	Em português	Exemplo
`ManyToOne`	Muitos para um	Muitas consultas pertencem a **um** paciente
`OneToOne`	Um para um	Cada consulta tem **um** pagamento
`ManyToMany`	Muitos para muitos	Um médico tem várias especialidades e cada especialidade tem vários médicos (tabela de junção `medico_especialidade`)

CID significa *Classificação Internacional de Doenças* (padrão da OMS). Cada doença tem um código; por exemplo, `J00` é o resfriado comum. É o valor guardado em `diagnosticoCid`.

Nomes: a entidade `Consulta` vira a tabela `consulta`. Campos em camelCase viram colunas em snake_case (ex.: `diagnosticoCid` ↔ `diagnostico_cid`, como visto na migração V6).

Fluxo de status completo da consulta. O diagrama da seção 4.1 não mostra o status **TRIADA**, mas ele é atribuído pelo `TriagemService` (seção 4.4) e aparece no fluxo 9.1. O ciclo completo é:

```
AGENDADA → CONFIRMADA → AGUARDANDO_TRIAGEM → TRIADA → REALIZADA
    ↓           ↓
CANCELADA    CANCELADA
```

Fluxo de status do pagamento: `PENDENTE → PAGO → ESTORNADO` (só se paga o que está PENDENTE; só se estorna o que está PAGO).

Ordem de prioridade da triagem (da mais urgente para a menos): `EMERGENCIA` > `ALTA` > `MEDIA` > `BAIXA`.

➕ ADIÇÃO — Exemplo ilustrativo: como uma entidade aparece em código

Simplificado para fins didáticos; não é o código exato do projeto.

```java
@Entity                          // esta classe vira uma tabela
@Data                            // Lombok gera getters e setters
public class Consulta {

    @Id @GeneratedValue
    private UUID id;             // coluna id (gerada automaticamente)

    @ManyToOne
    private Paciente paciente;   // muitas consultas -> um paciente

    @ManyToOne
    private Medico medico;       // muitas consultas -> um médico

    private LocalDateTime dataHora;
    private String status;       // AGENDADA, CONFIRMADA, ...
}
```

Cada linha da tabela `consulta` no banco corresponde a **um objeto** `Consulta` no Java.

4.2. Camada de DTOs (dto/)

O que são DTOs? DTO significa "Data Transfer Object" (Objeto de Transferência de Dados). São objetos simples que definem exatamente quais dados a API recebe e retorna. Eles existem por segurança: em vez de expor a entidade inteira (que pode conter dados sensíveis como senha), o DTO filtra apenas os campos necessários.

Analogia: Se a entidade é o seu documento completo (RG, CPF, endereço, tudo), o DTO é o crachá que mostra só o que é necessário para aquele contexto.

LoginRequest.java

```
Campos: email, senha
Uso: Recebe credenciais do usuário no endpoint /auth/login
```

LoginResponse.java

```
Campos: token, role
Uso: Retorna o JWT e a role do usuário após login bem-sucedido
```

RegisterRequest.java

```
Campos: nome, email, senha, cpf, dataNascimento, telefone
Uso: Dados para cadastro de novo paciente em /auth/register
```

ConsultaDTO.java

```
Campos: id, medicoId, pacienteId, dataHora, status, pagamentoId
Uso: Agendamento e listagem de consultas
Validações: dataHora @Future (deve ser no futuro), medicoId e pacienteId @NotNull
```

PacienteDTO.java

```
Campos: id, cpf, dataNascimento, telefone, nomeUsuario, nome, email, senha
Uso: Cadastro e listagem de pacientes
```

PacienteClinicoDTO.java

```
Campos: id, nome, cpf, dataNascimento, telefone, prontuarios[], receitas[], atestados[]
Uso: Visão clínica completa do paciente (prontuários, receitas, atestados)
```

MedicoDTO.java

```
Campos: id, crm, nomeUsuario, valorConsulta, especialidades[]
Uso: Listagem e detalhes de médicos
```

ProntuarioDTO.java

```
Campos: id, consultaId, diagnosticoCid, evolucaoClinica, condutaMedica, observacoes
Uso: Criação e visualização de prontuários
```

PagamentoDTO.java

```
Campos: id, consultaId, valor, status, formaPagamento
Uso: Visualização e operações de pagamento
```

TriagemDTO.java

```
Campos: id, consultaId, pressaoArterial, temperatura, frequenciaCardiaca, peso, altura, saturacao,
queixaPrincipal, prioridade
Uso: Registro de triagem pela equipe de enfermagem
```

ReceitaDTO.java

```
Campos: id, pacienteId, medicamento, dosagem, instrucoes, dataEmissao
Uso: Prescrição de medicamentos
```

AtestadoDTO.java

```
Campos: id, pacienteId, descricao, dias, dataEmissao
Uso: Emissão de atestados
```

AuditoriaDTO.java

```
Campos: id, usuarioId, usuarioEmail, acao, entidade, entidadeId, detalhes, dataHora
Uso: Visualização de logs de auditoria
```

FaturamentoRelatorioDTO.java

```
Campos: totalFaturadoDia, totalFaturadoMes, pagamentosPendentesCount, pagamentosRecebidosCount,
totalPendente
Uso: Relatório financeiro para o setor de faturamento
```

FuncionarioRequest.java

```
Campos: nome, email, senha, cargo, matricula, dataAdmissao
Uso: Criação de funcionários pelo admin
```

➕ ADIÇÃO — Validações e cuidados com DTOs

- `@NotNull` significa "campo obrigatório"; `@Future` significa "a data precisa ser no futuro". Se o pedido violar, a API recusa antes de chegar à regra de negócio.
- **Caminho de um dado:** o Controller recebe o **DTO**, o Service o converte em **entidade**, salva no banco e devolve outro **DTO**. A entidade completa nunca vai direto para a tela.
- **Atenção com senha:** o `PacienteDTO` possui o campo `senha` (usado no cadastro feito pela recepção). Campos de senha devem ser **somente de entrada**: nunca devem voltar nas respostas da API. *Confirmar no código* que a senha não é preenchida ao devolver um paciente.

➕ ADIÇÃO — Comparação: entidade x DTO

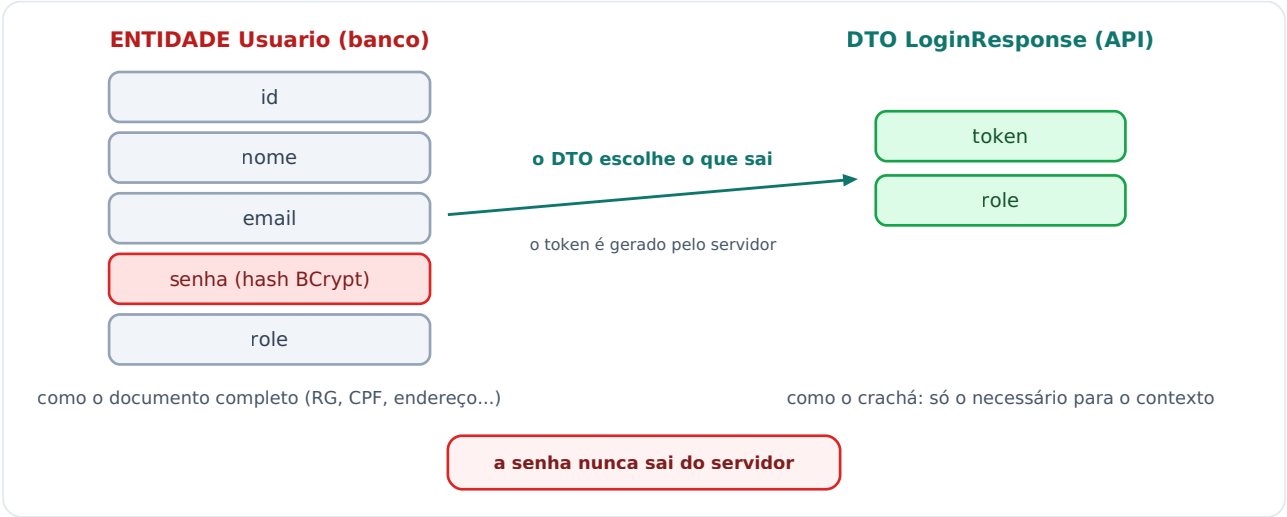


Figura 3 — A entidade guarda tudo; o DTO escolhe o que sai pela API

➕ ADIÇÃO — Exemplos de DTOs em JSON

Valores fictícios, apenas para ilustrar os campos descritos acima.

Cadastro de paciente (`RegisterRequest` , enviado em `POST /auth/register`):

```
{
  "nome": "Maria Souza",
  "email": "maria@email.com",
  "senha": "MinhaSenha123",
  "cpf": "123.456.789-09",
  "dataNascimento": "1990-05-20",
  "telefone": "(49) 99999-0000"
}
```

Login (`LoginRequest` , enviado em `POST /auth/login`) e a **resposta** (`LoginResponse`):

```
{ "email": "maria@email.com", "senha": "MinhaSenha123" }
```

```
{ "token": "eyJhbGciOiJIUzI1NiJ9...", "role": "PACIENTE" }
```

Agendamento (`ConsultaDTO`): ao enviar, só são preenchidos `medicoId` , `pacienteId` e `dataHora` ; a resposta traz também `id` , `status` e `pagamentoId` :

```
{ "medicoId": "3f2b8c10-...", "pacienteId": "a91c77de-...", "dataHora": "2026-10-15T10:00:00" }
```

```
{ "id": "7d4e...", "medicoId": "3f2b8c10-...", "pacienteId": "a91c77de-...",
  "dataHora": "2026-10-15T10:00:00", "status": "AGENDADA", "pagamentoId": "c5b1..." }
```

4.3. Camada de Repositórios (repository/)

O que são Repositórios? São interfaces que fazem a ponte entre o código Java e o banco de dados. O Spring Data JPA implementa automaticamente métodos baseados no nome do método.

Analogia: É como ter um bibliotecário que entende pedidos em português. Você fala "encontre por email" (`findByEmail`) e ele sabe exatamente como buscar no banco.

Métodos importantes (além dos padrões findById, findAll, save, delete):

Repositório	Método Personalizado	O que faz
UsuarioRepository	`findByEmail(String email)`	Busca usuário pelo e-mail
ConsultaRepository	`existsByMedicoIdAndDataHoraBetween(UUID, LocalDateTime, LocalDateTime)`	Verifica conflito de horário do médico
ConsultaRepository	`existsByPacienteIdAndDataHoraBetween(UUID, LocalDateTime, LocalDateTime)`	Verifica conflito de horário do paciente
ConsultaRepository	`findByDataHoraBetween(LocalDateTime, LocalDateTime)`	Lista consultas de um período
ConsultaRepository	`findByMedicoIdAndDataHoraBetween(UUID, LocalDateTime, LocalDateTime)`	Agenda do dia de um médico
PagamentoRepository	`findByConsultaId(UUID)`	Busca pagamento de uma consulta
ProntuarioRepository	`findByConsultaId(UUID)`	Busca prontuário de uma consulta

Repositório	Método Personalizado	O que faz
ProntuarioRepository	`findByConsultaPacienteId(UUID)`	Todos os prontuários de um paciente
TriagemRepository	`findByConsultaId(UUID)`	Busca triagem de uma consulta
MedicoRepository	`findByUsuarioEmail(String)`	Busca médico pelo e-mail do usuário
ReceitaRepository	`findByPacienteId(UUID)`	Todas receitas de um paciente
AtestadoRepository	`findByPacienteId(UUID)`	Todos atestados de um paciente

➕ ADIÇÃO — Como ler o nome de um método do repositório

O Spring transforma o nome do método em SQL. Exemplo: `existsByMedicoIdAndDataHoraBetween(...)` lê-se: **existe** (`exists`) uma consulta **por** (`By`) `Medico.id` **E** (`And`) `dataHora` **entre** (`Between`) dois horários? Se sim, há conflito.

Da mesma forma, `findByConsultaPacienteId` "navega" pelos relacionamentos: Prontuário → consulta → paciente → id.

Na prática dos conflitos de horário: como a verificação é de ±59 minutos, cada consulta ocupa cerca de **1 hora**. Se já existe uma às 10:00, tentar marcar às 10:30 conflita; marcar às 11:00 é aceito.

➕ ADIÇÃO — Exemplo ilustrativo de um repositório

Basta declarar o **nome** dos métodos; o Spring escreve o SQL sozinho:

```
public interface ConsultaRepository extends JpaRepository<Consulta, UUID> {

    // "existe consulta deste médico entre estes dois horários?"
    boolean existsByMedicoIdAndDataHoraBetween(UUID medicoId,
                                               LocalDateTime inicio,
                                               LocalDateTime fim);

    // "quais consultas existem neste período?"
    List<Consulta> findByDataHoraBetween(LocalDateTime inicio, LocalDateTime fim);
}
```

Exemplo numérico de conflito de horário (o Dr. Ana já tem consulta às 10:00):

Nova tentativa	Resultado	Motivo
09:00	Aceita	Está fora da janela 09:01 a 10:59
09:30	**Recusada**	Dentro da janela
10:30	**Recusada**	Dentro da janela
11:00	Aceita	Está fora da janela 09:01 a 10:59

4.4. Camada de Serviços (service/) — LÓGICA DE NEGÓCIO

Esta é a camada mais importante. Aqui vivem TODAS as regras do sistema.

AuthService.java — Autenticação

Método `register(RegisterRequest request)`:

1. Verifica se o e-mail já existe no banco → se sim, lança erro
2. Cria um novo `Usuario` com:
   - Nome, email do request
   - Senha criptografada com BCrypt (nunca salva em texto puro!)
   - Role = "PACIENTE" (todo novo cadastro é paciente)
3. Salva o usuário no banco
4. Cria um novo `Paciente` vinculado ao usuário
5. Salva o paciente
6. Gera um token JWT
7. Retorna o token e a role

Método `login(LoginRequest request)`:

1. Usa o AuthenticationManager do Spring para validar email + senha
2. Busca o usuário pelo email
3. Gera um token JWT
4. Retorna o token e a role

ConsultaService.java — Agendamento de Consultas

Método `agendar(ConsultaDTO dto)` — O mais complexo:

1. Valida que a data é no futuro (mínimo 30min de antecedência)
2. Busca o médico pelo ID → se não existe, erro
3. Busca o paciente pelo ID → se não existe, erro
4. **Verificação de ownership:** Se quem está logado é PACIENTE, só pode agendar para si mesmo
5. **Verificação de conflito de horário do médico:** Verifica se o médico já tem consulta no intervalo de ±59 minutos
6. **Verificação de conflito de horário do paciente:** Verifica se o paciente já tem consulta no mesmo intervalo
7. Cria a consulta com status "AGENDADA"
8. Cria automaticamente um Pagamento "PENDENTE" com o valor da consulta do médico
9. Envia notificação via RabbitMQ
10. Registra na auditoria
11. Retorna o DTO da consulta

Método `remarcar(UUID id, LocalDateTime novaDataHora)`:

1. Valida antecedência mínima de 30 minutos
2. Verifica que a consulta está AGENDADA
3. Verifica ownership (paciente só remarca a sua)
4. Valida conflitos de horário
5. Atualiza a data

6. Notifica e audita

Método `checkin(UUID id)`:

1. Busca a consulta
2. Verifica status válido (não pode ser CANCELADA ou REALIZADA)
3. Muda status para "AGUARDANDO_TRIAGEM"
4. Notifica e audita

Método `cancelar(UUID id)`:

1. Verifica ownership
2. Valida antecedência de 2 horas
3. Muda status para "CANCELADA"
4. Notifica e audita

Método `realizar(UUID id)`:

1. Verifica que não está cancelada
2. Verifica que o pagamento está "PAGO"
3. Muda status para "REALIZADA"
4. Audita

Método `agendaHoje()`:

1. Identifica o médico logado pelo email do JWT
2. Busca consultas do dia
3. Ordena por prioridade de triagem: EMERGENCIA > ALTA > MEDIA > BAIXA

PagamentoService.java — Gestão de Pagamentos

Método `pagar(UUID id)`:

1. Busca o pagamento pelo ID
2. Valida que está PENDENTE
3. Muda status para "PAGO"
4. Registra na auditoria

Método `estornar(UUID id)`:

1. Busca o pagamento
2. Valida que está PAGO
3. Muda status para "ESTORNADO"
4. Audita

Método `findById(UUID id)`:

1. Busca o pagamento
2. Se quem pede é PACIENTE, verifica que é seu (ownership check)

FaturamentoService.java — Relatórios Financeiros

Método `gerarRelatorio()`:

1. Busca TODOS os pagamentos do banco
2. Calcula:
    - Total faturado no dia (pagamentos PAGO com data de hoje)
    - Total faturado no mês (pagamentos PAGO do mês atual)

- Quantidade de pendentes e recebidos
- Total em reais pendente

AuditoriaService.java — Registro de Ações

Método `registrarAcao(String acao, String entidade, UUID entidadeId, String detalhes)`:

1. Pega o usuário logado do SecurityContext
2. Busca o `Usuario` completo pelo email
3. Cria um registro de `Auditoria` com todos os dados
4. Salva no banco
5. Se der erro, captura silenciosamente (nunca trava o fluxo principal)

TriagemService.java — Triagem de Enfermagem

Método `registrarTriagem(TriagemDTO dto)`:

1. Busca a consulta associada
2. Valida que o status é "AGUARDANDO_TRIAGEM"
3. Cria o registro com sinais vitais e prioridade
4. Muda status da consulta para "TRIADA"

PacienteService.java — Gestão de Pacientes

Inclui lógica para:

- Criação inline de paciente + usuário (pela recepção)
- Prescrição de receitas (`prescreverReceita`)
- Emissão de atestados (`emitirAtestado`)
- Visualização clínica completa (`mapToClinicoDTO`) com prontuários, receitas e atestados

NotificacaoListener.java — Consumidor de Mensagens RabbitMQ

Escuta a fila `notificacoes` e imprime no console. Aqui é onde, no futuro, se integraria com serviço de e-mail, SMS ou push notification.

➕ ADIÇÃO — Resumo de todas as regras de negócio em um só lugar

Ação	Regras principais	Quem executa	Notifica (RabbitMQ)	Audita
Agendar	Mín. 30 min de antecedência; sem conflito de horário (médico e paciente, ±59 min); paciente só agenda para si; cria pagamento PENDENTE	PACIENTE, RECEPCIONISTA, ADMIN	Sim	Sim
Remarcar	Mín. 30 min; consulta precisa estar AGENDADA; paciente só remarca a sua; revalida conflitos	PACIENTE, RECEPCIONISTA, ADMIN	Sim	Sim
Confirmar	AGENDADA → CONFIRMADA	PACIENTE	Sim (citado na seção 2.4)	Não detalhado
Check-in	Não pode estar CANCELADA nem REALIZADA; vira AGUARDANDO_TRIAGEM	RECEPCIONISTA, ADMIN	Sim	Sim

Ação	Regras principais	Quem executa	Notifica (RabbitMQ)	Audita
Triagem	Exige AGUARDANDO_TRIAGEM; vira TRIADA	TRIAGEM	Não detalhado	Não detalhado
Pagar	Exige PENDENTE; vira PAGO	FATURAMENTO, ADMIN, RECEPCIONISTA	Não	Sim
Estornar	Exige PAGO; vira ESTORNADO	FATURAMENTO, ADMIN	Não	Sim
Cancelar	Mín. 2 horas de antecedência; paciente só cancela a sua	PACIENTE, RECEPCIONISTA, ADMIN	Sim	Sim
Realizar	Não pode estar cancelada; pagamento precisa estar PAGO	MEDICO	Não	Sim

"Não detalhado" indica que o texto original não informa.

➕ ADIÇÃO — Serviços que existem, mas sem passo a passo no texto original

(Descrição inferida a partir das rotas, DTOs e permissões; o passo a passo exato deve ser confirmado no código.)

- **AdminService:** cria funcionários (`POST /admin/funcionarios`) a partir do `FuncionarioRequest` (nome, e-mail, senha, cargo, matrícula, data de admissão): cria o `Usuario` com a role do cargo e o registro de `Funcionario`.
- **MedicoService:** cadastra e lista médicos, com CRM, valor da consulta e especialidades.
- **EspecialidadeService:** cria e lista especialidades.
- **ExameService:** solicita, lista e detalha exames.
- **ProntuarioService:** cria/edita prontuários; no histórico, o médico vê todos e o paciente apenas os seus.
- **ConsultaService.confirmar:** muda AGENDADA para CONFIRMADA (somente PACIENTE).

➕ ADIÇÃO — Dois conceitos usados nas regras acima

- **Ownership (propriedade):** verificação de que o recurso pertence a quem está pedindo. Ex.: um paciente só enxerga e altera **as suas** consultas e pagamentos.
- **SecurityContext:** "memória" do Spring, durante uma requisição, sobre **quem** está logado e com quais permissões. É de lá que os serviços descobrem o usuário atual.

➕ ADIÇÃO — Comparação: como as camadas trabalham juntas

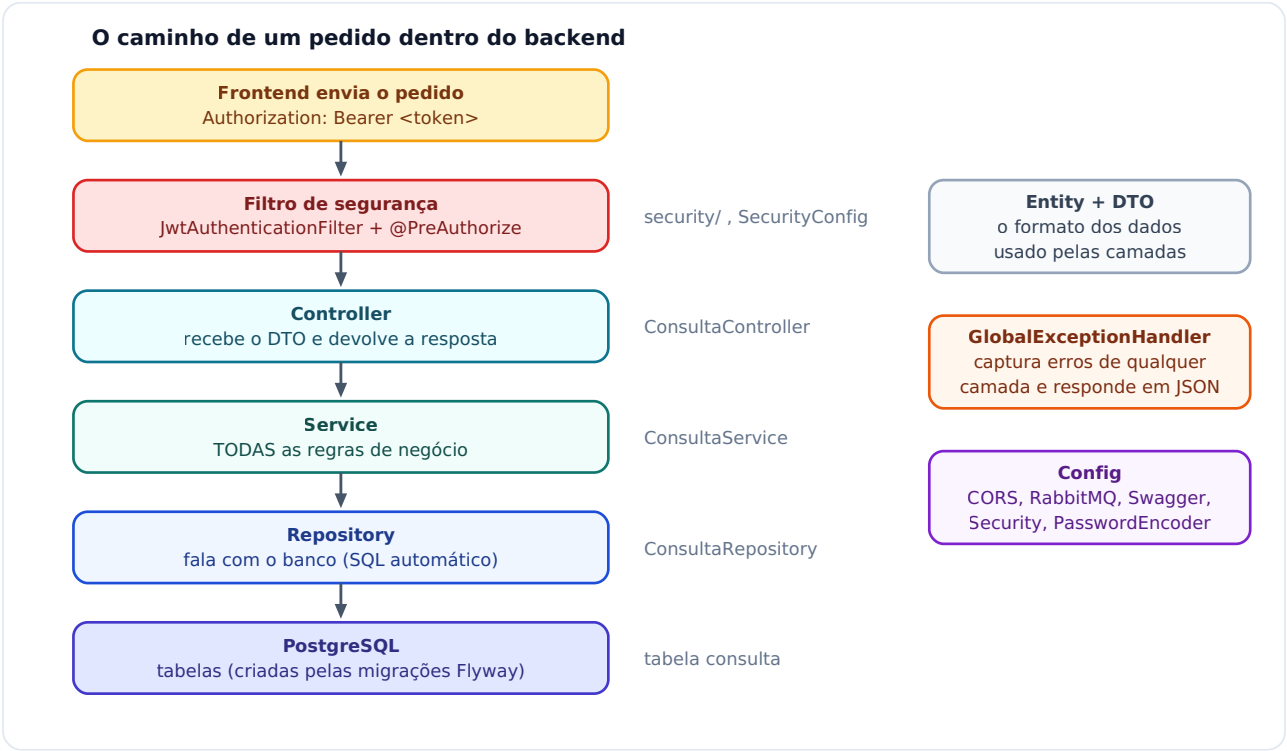


Figura 4 — O caminho de um pedido pelas camadas do backend

➕ ADIÇÃO — Pseudocódigo do método `agendar`

Os 11 passos da lista acima, lidos como código. *Os nomes dos métodos auxiliares são inventados para ilustrar; o que vale é a ordem das etapas.*

```
public ConsultaDTO agendar(ConsultaDTO dto) {
    validarAntecedenciaMinima(dto.getDataHora());              // 1. mínimo 30 min
    Medico medico = buscarMedico(dto.getMedicoId());           // 2. existe?
    Paciente paciente = buscarPaciente(dto.getPacienteId());   // 3. existe?
    verificarOwnership(paciente);                              // 4. paciente só para si
    verificarConflito(medico, paciente, dto.getDataHora());    // 5 e 6. ±59 min
    Consulta c = salvarConsulta(medico, paciente, dto, "AGENDADA");      // 7
    criarPagamentoPendente(c, medico.getValorConsulta());      // 8. PENDENTE
    notificar(c);                                              // 9. RabbitMQ
    auditoria.registrarAcao("AGENDAR_CONSULTA", "Consulta", c.getId(), "...");  // 10
    return paraDTO(c);                                         // 11
}
```

➕ ADIÇÃO — Exemplo: a agenda do dia ordenada por prioridade

Quatro pacientes agendados para hoje com o mesmo médico:

Paciente	Horário	Prioridade da triagem
Ana	09:00	BAIXA
Bruno	09:30	EMERGENCIA
Carla	08:00	MEDIA
Davi	10:00	ALTA

Ordem em que aparecem em `GET /api/consultas/medico/agenda-hoje`: **Bruno** (EMERGENCIA), **Davi** (ALTA), **Carla** (MEDIA), **Ana** (BAIXA). O horário marcado não manda: quem é mais grave vem antes. *(O texto original não informa o critério de desempate entre duas prioridades iguais.)*

➕ **ADIÇÃO — Exemplo numérico do relatório de faturamento**

Suponha que hoje existam estes pagamentos e que todos sejam de hoje:

Pagamento	Valor	Status
1	R$ 200,00	PAGO
2	R$ 150,00	PAGO
3	R$ 300,00	PENDENTE

O relatório (`FaturamentoRelatorioDTO`) ficaria: `totalFaturadoDia` = **350,00**; `totalFaturadoMes` = **350,00** (se não houver outros pagamentos no mês); `pagamentosRecebidosCount` = **2**; `pagamentosPendentesCount` = **1**; `totalPendente` = **300,00**.

4.5. Camada de Controllers (controller/) — ROTAS HTTP

Mapa COMPLETO de todas as rotas da API:

Autenticação (AuthController — /auth)

Método	Rota	Descrição	Quem pode
POST	`/auth/login`	Fazer login	Todos (público)
POST	`/auth/register`	Cadastrar paciente	Todos (público)

Admin (AdminController — /admin)

Método	Rota	Descrição	Quem pode
POST	`/admin/funcionarios`	Criar funcionário	ADMIN

Consultas (ConsultaController — /api/consultas)

Método	Rota	Descrição	Quem pode
POST	`/api/consultas`	Agendar consulta	PACIENTE, RECEPCIONISTA, ADMIN
GET	`/api/consultas`	Listar todas	ADMIN, RECEPCIONISTA
PUT	`/api/consultas/{id}/remarcar`	Remarcar	PACIENTE, RECEPCIONISTA, ADMIN
PUT	`/api/consultas/{id}/confirmar`	Confirmar	PACIENTE
PUT	`/api/consultas/{id}/checkin`	Check-in na recepção	RECEPCIONISTA, ADMIN
PUT	`/api/consultas/{id}/cancelar`	Cancelar	PACIENTE, RECEPCIONISTA, ADMIN
PUT	`/api/consultas/{id}/realizar`	Marcar como realizada	MEDICO

Método	Rota	Descrição	Quem pode
GET	`/api/consultas/hoje`	Agenda do dia	RECEPCIONISTA, ADMIN
GET	`/api/consultas/medico/agenda-hoje`	Agenda do médico logado	MEDICO

Pacientes (PacienteController — /api/pacientes)

Método	Rota	Descrição	Quem pode
POST	`/api/pacientes`	Criar paciente (recepção)	RECEPCIONISTA, ADMIN
GET	`/api/pacientes`	Listar todos	ADMIN, RECEPCIONISTA
GET	`/api/pacientes/{id}`	Detalhes de um paciente	Autenticado
GET	`/api/pacientes/{id}/clinico`	Visão clínica completa	MEDICO, PACIENTE
POST	`/api/pacientes/{id}/receitas`	Prescrever receita	MEDICO
POST	`/api/pacientes/{id}/atestados`	Emitir atestado	MEDICO

Médicos (MedicoController — /api/medicos)

Método	Rota	Descrição	Quem pode
POST	`/api/medicos`	Cadastrar médico	ADMIN
GET	`/api/medicos`	Listar todos	Autenticado
GET	`/api/medicos/{id}`	Detalhes	Autenticado

Prontuários (ProntuarioController — /api/prontuarios)

Método	Rota	Descrição	Quem pode
POST	`/api/prontuarios`	Criar/editar prontuário	MEDICO
GET	`/api/prontuarios/historico`	Histórico (médico vê todos, paciente só os seus)	MEDICO, PACIENTE
GET	`/api/prontuarios/{id}`	Detalhes de um prontuário	MEDICO, PACIENTE

Pagamentos (PagamentoController — /api/pagamentos)

Método	Rota	Descrição	Quem pode
POST	`/api/pagamentos/{id}/pagar`	Registrar pagamento	FATURAMENTO, ADMIN, RECEPCIONISTA
POST	`/api/pagamentos/{id}/estornar`	Estornar pagamento	FATURAMENTO, ADMIN
GET	`/api/pagamentos/{id}`	Ver pagamento	FATURAMENTO, ADMIN, RECEPCIONISTA, PACIENTE

Triagem (via ConsultaController)

Método	Rota	Descrição	Quem pode
POST	`/api/triagem`	Registrar triagem	TRIAGEM

Especialidades (EspecialidadeController — /api/especialidades)

Método	Rota	Descrição	Quem pode
POST	`/api/especialidades`	Criar especialidade	ADMIN
GET	`/api/especialidades`	Listar todas	Autenticado

Exames (ExameController — /api/exames)

Método	Rota	Descrição	Quem pode
POST	`/api/exames`	Solicitar exame	MEDICO
GET	`/api/exames`	Listar exames	MEDICO, ADMIN
GET	`/api/exames/{id}`	Detalhes	Autenticado

Faturamento (FaturamentoController — /api/faturamento)

Método	Rota	Descrição	Quem pode
GET	`/api/faturamento/relatorio`	Relatório financeiro	FATURAMENTO, ADMIN

Auditoria (AuditoriaController — /api/auditoria)

Método	Rota	Descrição	Quem pode
GET	`/api/auditoria?limite=100`	Últimos logs do sistema	ADMIN

Endpoints de Infraestrutura (públicos)

Método	Rota	Descrição
GET	`/actuator/health`	Status de saúde da aplicação
GET	`/actuator/prometheus`	Métricas para Prometheus
GET	`/swagger-ui.html`	Documentação interativa da API
GET	`/v3/api-docs`	Especificação OpenAPI em JSON

➕ ADIÇÃO — Como ler as tabelas de rotas

- **GET** lê dados, **POST** cria ou executa uma ação, **PUT** altera, **DELETE** remove.
- `{id}` é um espaço reservado: em `/api/consultas/{id}/cancelar`, troca-se `{id}` pelo UUID da consulta.
- **Códigos de resposta HTTP mais comuns:**

Código	Significado
200 / 201	Deu certo / criado com sucesso
400	Dados inválidos no pedido

Código	Significado
401	Não autenticado (sem token, token inválido ou expirado)
403	Autenticado, mas **sem permissão** para aquela ação (citado na seção 5.2)
404	Recurso não encontrado

Os códigos exatos devolvidos em cada erro de regra de negócio dependem do `GlobalExceptionHandler`.

➕ ADIÇÃO — Pacotes `config/`, `exception/` e `test/` explicados

Arquivo	Para que serve
`AppConfig`	Configuração geral; define o `PasswordEncoder` (BCrypt) usado nas senhas
`CorsConfig`	Libera o frontend (`localhost:5173`) a chamar o backend (`localhost:8080`). Como as portas são diferentes, o navegador as trata como origens diferentes e bloqueia a chamada, a menos que o backend autorize (isso é o CORS)
`OpenApiConfig`	Configura a documentação Swagger
`RabbitMQConfig`	Declara a fila `notificacoes`
`SecurityConfig`	Define quais rotas são públicas, registra o filtro JWT e ativa o `@PreAuthorize`
`GlobalExceptionHandler` / `ErrorResponse`	Ponto central que captura erros e devolve uma resposta JSON padronizada, em vez de mostrar erro técnico. Campos exatos: ver `ErrorResponse.java`
Testes (inferido pelos nomes)	`MedflowApplicationTests` (o sistema sobe?), `SecurityIntegrationTest` (rotas protegidas), `AuthServiceTest` e `ConsultaServiceTest` (regras dos serviços). Rodam com `mvn test`

➕ ADIÇÃO — Perfis (profiles) e variáveis de ambiente

O `application.yml` traz a configuração base; `application-dev.yml` e `application-prod.yml` sobrescrevem o que muda em desenvolvimento e em produção. O `docker-compose.yml` ativa o perfil `dev` e passa por **variáveis de ambiente** os dados de conexão do banco e do RabbitMQ. Em produção, use o perfil `prod` e entregue segredos por variáveis de ambiente (ver Parte 13).

Dockerfile do backend (não detalhado no original): normalmente é feito em duas etapas: (1) o Maven compila e gera o `.jar`; (2) uma imagem leve com Java executa esse `.jar` na porta 8080.

➕ ADIÇÃO — Exemplos de chamadas à API

Valores fictícios. Os campos vêm dos DTOs da seção 4.2; os nomes exatos e quais campos o sistema preenche sozinho devem ser confirmados no Swagger.

1. Login pelo terminal (curl):

```
curl -X POST http://localhost:8080/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@medflow.com","senha":"admin"}'
```

2. Usando o token nas próximas chamadas:

```
TOKEN="cole-aqui-o-token-recebido"
curl http://localhost:8080/api/medicos \
  -H "Authorization: Bearer $TOKEN"
```

3. Triagem (`POST /api/triagem` , role TRIAGEM):

```
{
  "consultaId": "7d4e...",
  "pressaoArterial": "120/80",
  "temperatura": 36.7,
  "frequenciaCardiaca": 78,
  "peso": 72.5,
  "altura": 1.75,
  "saturacao": 98,
  "queixaPrincipal": "Dor de cabeça há 2 dias",
  "prioridade": "MEDIA"
}
```

4. Prontuário (`POST /api/prontuarios` , role MEDICO):

```
{
  "consultaId": "7d4e...",
  "diagnosticoCid": "J00",
  "evolucaoClinica": "Paciente com coriza e dor de garganta há 3 dias, sem febre.",
  "condutaMedica": "Repouso, hidratação e sintomáticos.",
  "observacoes": "Retornar se houver febre."
}
```

5. Receita (`POST /api/pacientes/{id}/receitas` , role MEDICO). O paciente vai na URL; `dataEmissao` provavelmente é preenchida pelo sistema:

```
{ "medicamento": "Paracetamol", "dosagem": "500mg", "instrucoes": "1 comprimido a cada 8 horas
por 3 dias" }
```

6. Atestado (`POST /api/pacientes/{id}/atestados` , role MEDICO):

```
{ "descricao": "Afastamento por quadro gripal", "dias": 2 }
```

7. Pagamento (`POST /api/pagamentos/{id}/pagar` , role FATURAMENTO, ADMIN ou RECEPCIONISTA):

```
curl -X POST http://localhost:8080/api/pagamentos/c5b1.../pagar \
  -H "Authorization: Bearer $TOKEN"
```

8. Criar funcionário (`POST /admin/funcionarios` , role ADMIN):

```
{
  "nome": "João Lima",
  "email": "joao@medflow.com",
  "senha": "SenhaForte#2026",
  "cargo": "RECEPCIONISTA",
  "matricula": "F-0042",
  "dataAdmissao": "2026-09-01"
}
```

9. Exemplo de resposta de erro (formato **ilustrativo**; o real está em `ErrorResponse.java`):

```
{ "status": 403, "mensagem": "Acesso negado" }
```

➕ ADIÇÃO — Exemplo ilustrativo de um controller

```
@PostMapping("/api/consultas")
@PreAuthorize("hasAnyRole('PACIENTE','RECEPCIONISTA','ADMIN')")   // quem pode
public ConsultaDTO agendar(@Valid @RequestBody ConsultaDTO dto) { // @Valid aplica @Future,
@NotNull
    return consultaService.agendar(dto);                          // a regra mora no service
}
```

Note que o controller quase não tem lógica: ele **só recebe, confere a permissão e repassa** ao service.

PARTE 5 — SEGURANÇA: COMO FUNCIONA A PROTEÇÃO

5.1. Fluxo de Autenticação (Login)

```
1. Usuário envia POST /auth/login com { email, senha }
2. AuthService valida as credenciais com BCrypt
3. JwtService gera um token JWT contendo:
   - Subject: email do usuário
   - Role: papel do usuário
   - Emitido em: timestamp atual
   - Expira em: 24 horas depois
4. Sistema retorna { token: "eyJhbG...", role: "PACIENTE" }
5. Frontend salva o token no localStorage do navegador
```

5.2. Fluxo de Autorização (Acessar recurso protegido)

```
1. Frontend faz requisição com header: Authorization: Bearer eyJhbG...
2. JwtAuthenticationFilter intercepta a requisição
3. Extrai o token do header
4. JwtService valida:
   - Assinatura (não foi adulterado?)
   - Expiração (ainda é válido?)
5. Extrai o email do token
6. Busca o usuario completo no banco
7. Cria um contexto de segurança com as authorities (roles)
8. A requisição segue para o Controller
9. @PreAuthorize verifica se a role do usuário permite a ação
10. Se sim: executa. Se não: retorna 403 Forbidden.
```

5.3. Tabela de Permissões por Role

Funcionalidade	ADMIN	MEDICO	PACIENTE	RECEPCIONISTA	TRIAGEM	FATURAMENTO
Criar funcionário	✅	❌	❌	❌	❌	❌
Criar médico	✅	❌	❌	❌	❌	❌
Agendar consulta	✅	❌	✅ (só p/ si)	✅	❌	❌
Remarcar consulta	✅	❌	✅ (só a sua)	✅	❌	❌
Check-in	✅	❌	❌	✅	❌	❌
Cancelar consulta	✅	❌	✅ (só a sua)	✅	❌	❌
Realizar consulta	❌	✅	❌	❌	❌	❌
Fazer triagem	❌	❌	❌	❌	✅	❌
Criar prontuário	❌	✅	❌	❌	❌	❌
Ver prontuário	❌	✅ (todos)	✅ (só os seus)	❌	❌	❌
Prescrever receita	❌	✅	❌	❌	❌	❌

Funcionalidade	ADMIN	MEDICO	PACIENTE	RECEPCIONISTA	TRIAGEM	FATURAMENTO
Emitir atestado	❌	✅	❌	❌	❌	❌
Pagar consulta	✅	❌	❌	✅	❌	✅
Estornar pagamento	✅	❌	❌	❌	❌	✅
Relatório faturamento	✅	❌	❌	❌	❌	✅
Ver auditoria	✅	❌	❌	❌	❌	❌
Swagger/API docs	✅	✅	✅	✅	✅	✅

5.4. Senhas e Criptografia

- Senhas NUNCA são salvas em texto puro. São criptografadas com **BCrypt** antes de ir para o banco.
- BCrypt é um algoritmo de hash de mão única: é impossível reverter o hash para descobrir a senha original.
- O token JWT é assinado com uma chave secreta definida no `application.yml`.

➕ ADIÇÃO — Permissões complementares (derivadas das rotas da seção 4.5)

A tabela 5.3 não cobre todas as rotas. As que faltam:

Funcionalidade	Quem pode
Confirmar consulta	PACIENTE
Listar todas as consultas	ADMIN, RECEPCIONISTA
Agenda geral do dia	RECEPCIONISTA, ADMIN
Agenda do dia do médico logado	MEDICO
Cadastrar paciente (recepção)	RECEPCIONISTA, ADMIN
Listar pacientes	ADMIN, RECEPCIONISTA
Ver detalhes de um paciente	Qualquer usuário autenticado
Visão clínica do paciente	MEDICO, PACIENTE
Cadastrar médico	ADMIN
Listar / ver médicos	Qualquer usuário autenticado
Criar especialidade	ADMIN (listar: qualquer autenticado)
Solicitar exame	MEDICO
Listar exames	MEDICO, ADMIN (detalhes: qualquer autenticado)
Ver um pagamento	FATURAMENTO, ADMIN, RECEPCIONISTA e PACIENTE (só o seu)
Histórico e detalhes de prontuário	MEDICO, PACIENTE

Observação sobre o Swagger: na tabela 5.3 ele aparece liberado para todas as roles, enquanto na seção 4.5 os endpoints de infraestrutura estão como **públicos** (sem login). Para produção, veja a Parte 13.

➕ ADIÇÃO — Comparação: o caminho do login e das chamadas protegidas

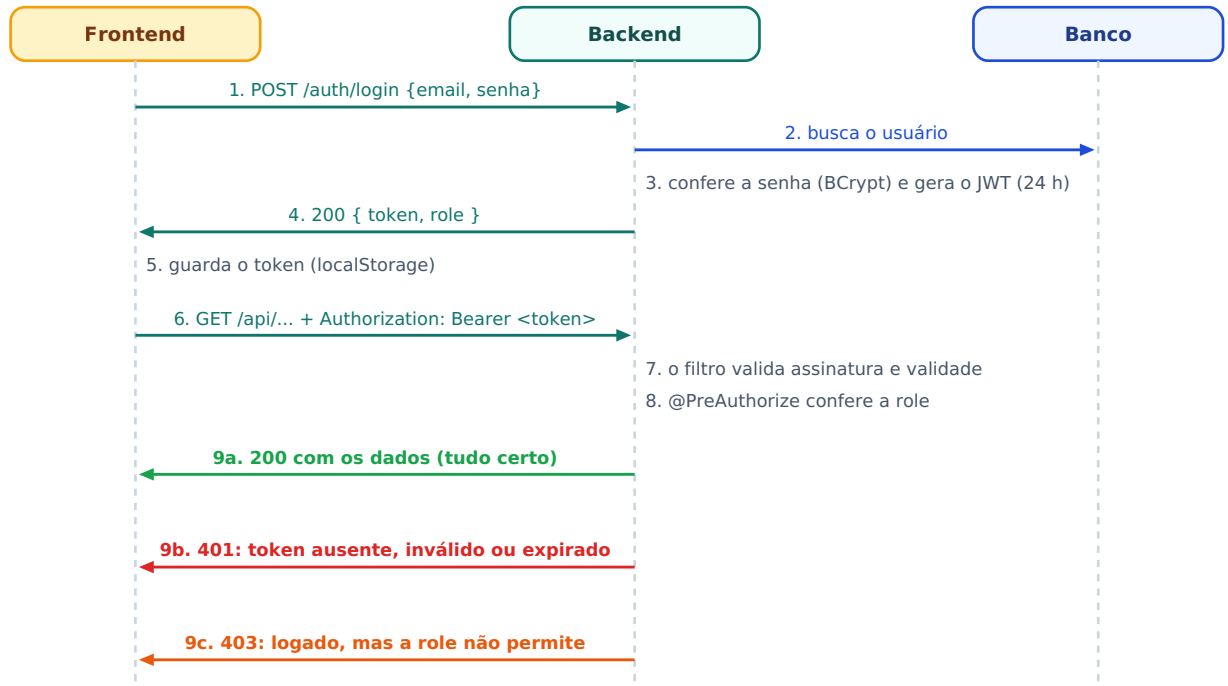


Figura 5 — Sequência de login e acesso a um recurso protegido (401 e 403)

➕ ADIÇÃO — Como é "por dentro" um token JWT

Um JWT tem três partes separadas por ponto: `cabeçalho.conteúdo.assinatura`. O **conteúdo** (*payload*) é apenas texto codificado, **não é secreto**; o que impede a falsificação é a **assinatura**, feita com a chave secreta. Um conteúdo típico, de acordo com a seção 5.1:

```
{ "sub": "maria@email.com", "role": "PACIENTE", "iat": 1790000000, "exp": 1790086400 }
```

`sub` é o e-mail, `role` é o papel, `iat` é quando foi emitido e `exp` é quando expira. Aqui `exp - iat = 86400` segundos, ou seja, **24 horas**. *(Os nomes exatos dos campos estão em `JwtService.java`.)* Por ser legível, **nunca coloque informações sensíveis dentro do token** e não cole tokens reais em sites de terceiros.

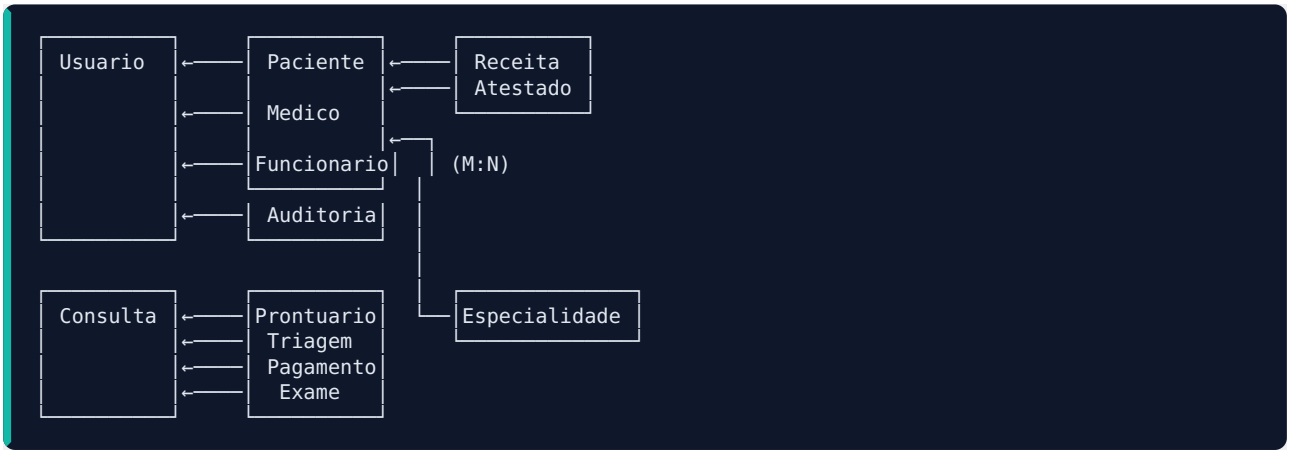
➕ ADIÇÃO — Cenários práticos: o que o sistema faz em cada situação

#	Cenário	Resultado	Por quê
1	Paciente tenta agendar consulta para **outro** paciente	Recusado	Verificação de ownership
2	Recepcionista tenta criar um prontuário	**403**	Só o MEDICO pode
3	Recepcionista tenta estornar um pagamento	**403**	Só FATURAMENTO e ADMIN
4	Faturamento estorna um pagamento PAGO	Permitido	Role autorizada e status válido
5	Paciente cancela uma consulta que começa em 1 hora	Recusado	Mínimo de 2 horas

#	Cenário	Resultado	Por quê
6	Paciente remarca para daqui a 20 minutos	Recusado	Mínimo de 30 minutos
7	Médico marca como realizada uma consulta com pagamento PENDENTE	Recusado	Exige pagamento PAGO
8	Check-in de uma consulta CANCELADA	Recusado	Status inválido para check-in
9	Triagem de uma consulta ainda AGENDADA	Recusada	Exige AGUARDANDO_TRIAGEM
10	Uso de um token emitido há 25 horas	**401**	Token expirado (validade de 24 h)
11	Dois agendamentos do mesmo médico às 10:00 e 10:30	O segundo é recusado	Conflito (janela de ±59 min)
12	Requisição sem o header `Authorization` em rota protegida	**401**	Sem identificação

PARTE 6 — BANCO DE DADOS: ESTRUTURA DAS TABELAS

6.1. Diagrama de Relacionamento



6.2. Migrações SQL (na ordem de execução)

V1 — Tabelas Iniciais

Cria: `usuario`, `paciente`, `medico`, `consulta`, `prontuario`, `pagamento`, `exame`, `especialidade`

V2 — Funcionários

Cria: `funcionario` (para recepcionistas, triagem, faturamento)

V3 — Especialidade Médica (M:N)

Cria: `medico_especialidade` (tabela de junção muitos-para-muitos)

V4 — Limpeza

Remove coluna `especialidade` da tabela `medico` (migrada para M:N)

V5 — Triagem

Cria: `triagem` (sinais vitais, prioridade)

V6 — Receitas e Atestados

Cria: `receita`, `atestado`. Adiciona colunas `diagnostico_cid`, `evolucao_clinica`, `conduta_medica` ao `prontuario`

V7 — Auditoria

Cria: `auditoria` (log de ações do sistema)

➕ ADIÇÃO — Como ler o diagrama de relacionamento

Cada seta `←——` indica "aponta para". `Paciente`, `Medico`, `Funcionario` e `Auditoria` apontam para `Usuario` (cada um tem uma **chave estrangeira**, ou FK, que guarda o `id` do usuário). `Receita` e `Atestado` apontam para `Paciente`. `Prontuario`, `Triagem`, `Pagamento` e `Exame` apontam para `Consulta`, a tabela central. `Medico` e `Especialidade` se ligam por uma tabela de junção (M:N).

➕ ADIÇÃO — Consultando o banco e conferindo as migrações

Depois de entrar com `docker exec -it medflow-db psql -U medflow -d medflow` (Parte 10):

Comando	O que faz
`\dt`	Lista todas as tabelas
`\d consulta`	Mostra as colunas da tabela `consulta`
`SELECT * FROM flyway_schema_history;`	Mostra quais migrações (V1 a V7...) já foram executadas
`\q`	Sai do psql

➕ ADIÇÃO — Exemplos de consultas SQL para explorar o banco

Dentro do `psql`, termine cada comando com `;`. *Os nomes das colunas são inferidos do padrão camelCase → snake_case; confira com `\d auditoria` antes de usar.*

```
-- Quantas consultas existem em cada status?
SELECT status, COUNT(*) FROM consulta GROUP BY status;

-- Pagamentos que ainda não foram pagos
SELECT * FROM pagamento WHERE status = 'PENDENTE';

-- As 10 últimas ações registradas na auditoria
SELECT usuario_email, acao, entidade, data_hora
FROM auditoria
ORDER BY data_hora DESC
LIMIT 10;
```

O primeiro exemplo devolve algo como: `AGENDADA | 3`, `CONFIRMADA | 1`, `REALIZADA | 5`. É uma forma rápida de "ver" o ciclo de vida da seção 9.2 acontecendo nos dados.

PARTE 7 — FRONTEND: COMO FUNCIONA A INTERFACE

7.1. Tecnologias do Frontend

Tecnologia	Versão	Função
React	18	Biblioteca de UI (construir telas)
TypeScript	-	Linguagem com tipos (menos erros)
Vite	-	Build tool (compilação rápida)
Tailwind CSS	3	Framework de CSS (estilização rápida)
Axios	-	Cliente HTTP (comunicação com backend)
React Router DOM	6	Navegação entre páginas sem recarregar

7.2. Páginas

Login.tsx

- Formulário com email e senha
- Chama POST `/auth/login`
- Salva o token no contexto e localStorage
- Redireciona para o Dashboard

Register.tsx

- Formulário com nome, email, senha, CPF, data de nascimento, telefone
- Chama POST `/auth/register`
- Cria o paciente e faz login automático

Dashboard.tsx

- Painel principal após login
- Mostra informações baseadas na role do usuário

Medicos.tsx

- Lista todos os médicos
- Mostra CRM, especialidades e valor da consulta

MinhasConsultas.tsx

- Lista as consultas do paciente logado
- Permite ver status e detalhes

7.3. Componentes de Infraestrutura

AuthContext.tsx

- Gerencia o estado de autenticação globalmente
- Armazena token e role
- Oferece funções `login()` e `logout()`
- Persiste no localStorage (não perde ao recarregar a página)

ProtectedRoute.tsx

- Componente que envolve páginas protegidas
- Se não está logado, redireciona para `/login`

Layout.tsx

- Layout com menu lateral (sidebar)
- Adapta os itens de menu baseado na role do usuário

axios.ts

- Configura o Axios com baseURL apontando para o backend
- Intercepta todas as requisições para adicionar o header `Authorization: Bearer <token>` automaticamente

7.4. Build e Deploy

No Dockerfile do frontend:

1. **Estágio de build:** Usa Node.js para instalar dependências e compilar (`npm run build`)
2. **Estágio de produção:** Copia os arquivos compilados para o Nginx
3. O `nginx.conf` configura:
   - Serve arquivos estáticos na porta 80
   - Redireciona `/api/*` para o backend (proxy reverso)
   - Suporta SPA (Single Page Application) com fallback para `index.html`

➕ ADIÇÃO — O que o frontend cobre hoje

Pela estrutura descrita (Parte 3 e seção 7.2), o frontend tem **5 páginas**: Login, Cadastro, Dashboard, Médicos e Minhas Consultas. Funcionalidades como triagem, prontuário, receitas, atestados, faturamento e auditoria existem no **backend** e nas rotas da API, mas **não têm tela própria** listada no documento. Hoje elas são usadas pelo Swagger ou por ferramentas de API. Telas para essas funções são a evolução natural do projeto.

➕ ADIÇÃO — Conceitos de React usados

Conceito	Significado simples
Componente	Peça reutilizável da tela (um botão, um menu, uma página)
Estado	Dados que, ao mudar, atualizam a tela sozinhos
Contexto (`AuthContext`)	Estado compartilhado por todas as telas (quem está logado)

Conceito	Significado simples
Rota protegida	Página que só abre se houver login
Interceptor (Axios)	Código que "carimba" todo pedido com o token antes de enviar
SPA	O site não recarrega ao navegar; o React troca o conteúdo da tela

O token no `localStorage` tem implicações de segurança (Parte 13).

PARTE 8 — INFRAESTRUTURA: DOCKER, KUBERNETES E MONITORAMENTO

8.1. Docker Compose (docker-compose.yml)

O arquivo `docker-compose.yml` define 6 serviços:

Serviço `db` (PostgreSQL)

- Porta: 5432
- Volume persistente: `medflow-postgres-data` (dados não se perdem ao reiniciar)
- Healthcheck: verifica se o PostgreSQL está pronto antes de liberar outros serviços

Serviço `rabbitmq` (RabbitMQ)

- Porta AMQP: 5672, Porta Web: 15672
- Usa tmpfs (armazenamento em memória) para melhor performance
- Healthcheck com timeout de 30s no start (demora a subir)

Serviço `backend` (Spring Boot)

- Porta: 8080
- Depende de `db` e `rabbitmq` estarem saudáveis
- Variáveis de ambiente configuram conexão com DB e RabbitMQ
- Profile ativo: `dev`

Serviço `frontend` (Nginx + React)

- Porta: 5173 → 80 (mapeada para 5173 no host por conveniência)
- Depende do backend estar rodando

Serviço `prometheus` (Monitoramento)

- Porta: 9090
- Monta o arquivo `prometheus.yml` como configuração
- Coleta métricas do backend a cada 15 segundos

Serviço `grafana` (Dashboards)

- Porta: 3000
- Depende do Prometheus
- Credenciais: admin/admin

Como subir tudo:

```
docker compose up -d
```

Este único comando sobe os 6 containers na ordem correta.

8.2. Kubernetes (k8s/)

Os arquivos Kubernetes são para deploy em produção (nuvem). Cada arquivo define:

backend.yaml

- Deployment: 2 réplicas do backend
- Service: Expõe na porta 8080
- ConfigMap: Variáveis de ambiente

frontend.yaml

- Deployment: 2 réplicas do frontend
- Service: Expõe na porta 80
- Pode ser configurado como LoadBalancer para acesso externo

postgres.yaml

- StatefulSet: 1 réplica (banco de dados não escala horizontalmente)
- PersistentVolumeClaim: 5GB de armazenamento persistente
- Service: Expõe na porta 5432 internamente

rabbitmq.yaml

- Deployment: 1 réplica
- Service: Portas 5672 e 15672

8.3. Prometheus (prometheus/prometheus.yml)

Configuração que define:

- Intervalo de scraping: 15 segundos
- Target: `backend:8080` (endpoint `/actuator/prometheus`)
- Métricas coletadas: HTTP requests, JVM memory, database connections, etc.

➕ ADIÇÃO — Conceitos de Docker usados neste arquivo

Termo	Significado simples
Imagem	O "molde" (receita pronta) de um container
Container	Uma imagem em execução
Volume	Pasta que **sobrevive** ao container (por isso o banco não perde dados ao reiniciar)
`5173:80`	Porta do seu computador **:** porta dentro do container
`depends_on` + healthcheck	"Só suba o backend depois que o banco estiver realmente pronto"
Rede do Compose	Os containers se falam pelo **nome do serviço**: o backend acessa `db:5432` e o Prometheus acessa `backend:8080`

Termo	Significado simples
`tmpfs`	Armazenamento em **memória RAM**: no RabbitMQ é rápido, mas mensagens ainda não entregues **são perdidas** se ele reiniciar

➕ ADIÇÃO — Glossário do Kubernetes

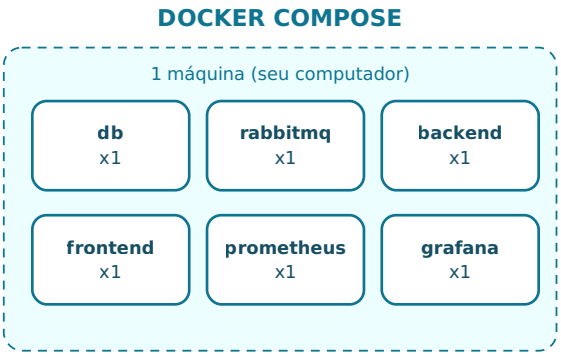
Termo	Significado simples
Réplica	Cópia idêntica de um serviço rodando em paralelo (o backend tem 2)
Deployment	Descreve "quero N cópias deste container sempre rodando"
Service	Endereço estável que distribui o tráfego entre as réplicas
ConfigMap	Variáveis de configuração **não secretas**
StatefulSet	Igual ao Deployment, mas para quem guarda dados (o banco)
PersistentVolumeClaim	Pedido de disco permanente (5 GB para o Postgres)
LoadBalancer	Ponto de acesso externo que reparte o tráfego

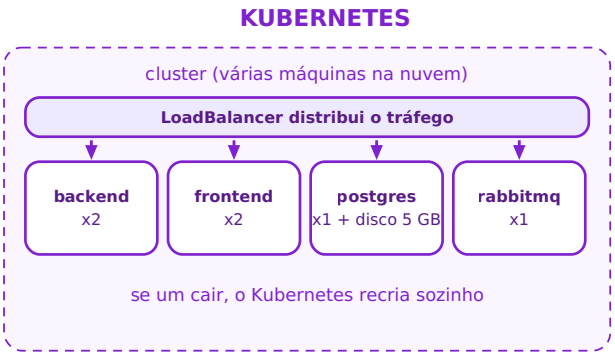
Senhas e chaves devem ir em **Secrets** do Kubernetes, não em ConfigMap (Parte 13).

➕ ADIÇÃO — Grafana sem dados?

Se o Grafana abrir mas os gráficos estiverem vazios, confira se o Prometheus está cadastrado como **fonte de dados** (*Data source*), com a URL `http://prometheus:9090` (o nome do serviço dentro da rede Docker).

➕ ADIÇÃO — Comparação: Docker Compose x Kubernetes





Comparação	Compose	Kubernetes
Indicado para	desenvolvimento e testes	produção
Se um container cair	você reinicia	reinicia automaticamente
Mais acessos	sobe na mão ou com mais containers	cria mais cópias sozinho
Atualizar sem parar	não	sim (rolling update)
Complexidade	baixa (1 arquivo)	maior (vários arquivos)

Figura 6 — Docker Compose (desenvolvimento) comparado ao Kubernetes (produção)

PARTE 9 — FLUXOS COMPLETOS DO SISTEMA

9.1. Fluxo Completo: Paciente faz consulta do início ao fim

```
1. PACIENTE acessa o site e clica em "Cadastrar"
   → POST /auth/register
   → Sistema cria Usuario (role=PACIENTE) + Paciente
   → Retorna token JWT

2. PACIENTE faz login
   → POST /auth/login
   → Retorna token JWT

3. PACIENTE agenda consulta
   → POST /api/consultas
   → Sistema valida conflitos de horário
   → Cria Consulta (status=AGENDADA)
   → Cria Pagamento (status=PENDENTE)
   → Envia notificação via RabbitMQ
   → Registra na auditoria

4. PACIENTE confirma a consulta
   → PUT /api/consultas/{id}/confirmar
   → Status muda para CONFIRMADA

5. PACIENTE chega na clínica, RECEPCIONISTA faz check-in
   → PUT /api/consultas/{id}/checkin
   → Status muda para AGUARDANDO_TRIAGEM

6. ENFERMEIRO DE TRIAGEM registra sinais vitais
   → POST /api/triagem
   → Registra pressão, temperatura, peso, etc.
   → Define prioridade: BAIXA, MEDIA, ALTA, EMERGENCIA
   → Status da consulta muda para TRIADA

7. FATURAMENTO registra pagamento
   → POST /api/pagamentos/{id}/pagar
   → Status do pagamento muda para PAGO
   → Registra na auditoria

8. MÉDICO atende o paciente (na agenda do dia, ordenada por prioridade)
   → GET /api/consultas/medico/agenda-hoje
   → Pacientes com EMERGENCIA aparecem primeiro

9. MÉDICO registra o prontuário
   → POST /api/prontuarios
   → Registra diagnóstico CID, evolução clínica, conduta médica

10. MÉDICO prescreve medicamento
    → POST /api/pacientes/{id}/receitas
    → Registra medicamento, dosagem, instruções

11. MÉDICO emite atestado (se necessário)
    → POST /api/pacientes/{id}/atestados
    → Registra descrição e dias de afastamento

12. MÉDICO marca consulta como realizada
    → PUT /api/consultas/{id}/realizar
    → Status muda para REALIZADA (só funciona se pagamento estiver PAGO)

13. ADMIN consulta a auditoria
    → GET /api/auditoria
    → Vê todo o rastro de quem fez o quê, quando

14. FATURAMENTO consulta relatório
```

```
    → GET /api/faturamento/relatorio
    → Vê total faturado no dia, mês, pendentes
```

➕ ADIÇÃO — 9.2. Ciclo de vida completo dos status

```
CONSULTA
AGENDADA ──► CONFIRMADA ──► AGUARDANDO_TRIAGEM ──► TRIADA ──► REALIZADA
    │             │            (check-in)          (triagem)   (médico; exige
    │             │                                            pagamento PAGO)
    └─────────────┴──► CANCELADA   (até 2 horas antes)

PAGAMENTO (criado junto com a consulta)
PENDENTE ──► PAGO ──► ESTORNADO
```

O check-in também é aceito a partir de AGENDADA, pois a regra só bloqueia CANCELADA e REALIZADA (seção 4.4).

➕ ADIÇÃO — 9.3. Fluxos alternativos

Situação	O que acontece
Paciente quer outro horário	`PUT /api/consultas/{id}/remarcar` (só se AGENDADA, com 30 min de antecedência e sem conflitos)
Paciente desiste	`PUT /api/consultas/{id}/cancelar` (até 2 horas antes); status CANCELADA
Pagamento feito por engano	`POST /api/pagamentos/{id}/estornar` (FATURAMENTO ou ADMIN); status ESTORNADO
Paciente grave	A triagem marca `EMERGENCIA` e ele aparece primeiro na agenda do médico
Médico tenta finalizar sem pagamento	`realizar` é recusado enquanto o pagamento não estiver PAGO

➕ ADIÇÃO — 9.4. A jornada técnica de uma ação (exemplo: agendar consulta)

```
[Tela React] clique em "Agendar"
      │  axios.ts acrescenta  Authorization: Bearer <token>
      ▼
[Backend :8080]
      ├─ JwtAuthenticationFilter   token válido? quem é o usuário?
      ├─ @PreAuthorize             essa role pode agendar?
      ├─ ConsultaController        recebe o ConsultaDTO
      └─ ConsultaService.agendar   regras: 30 min, conflitos, ownership
            ├─ Repositórios ──► PostgreSQL (consulta + pagamento PENDENTE)
            ├─ RabbitMQ: fila "notificacoes" ──► NotificacaoListener
            └─ AuditoriaService ──► tabela auditoria
      ▼
resposta JSON (ConsultaDTO) ──► a tela se atualiza
```

➕ ADIÇÃO — 9.5. Notificações: quem envia e quem recebe

O **produtor** é o `ConsultaService` (ao agendar, remarcar, cancelar, confirmar ou fazer check-in): ele deixa uma mensagem na fila `notificacoes` e segue em frente. O **consumidor** é o `NotificacaoListener`, que lê a fila e hoje só imprime no console. A vantagem é que um envio de e-mail lento no futuro **não trava** o agendamento, pois o processo é assíncrono.

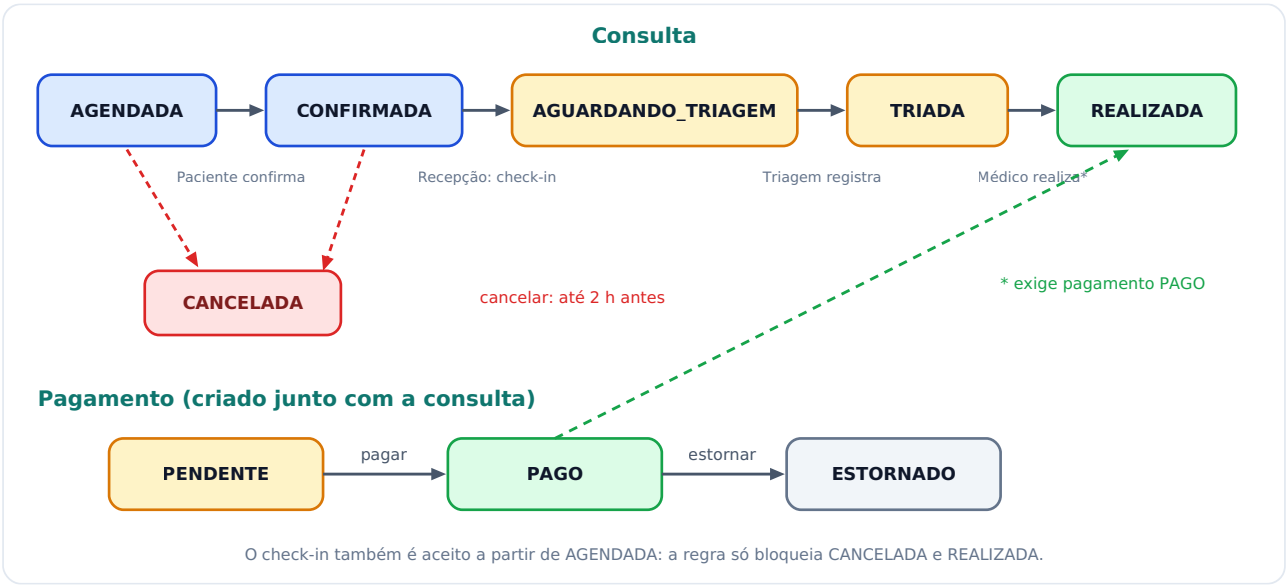


Figura 7 — Ciclo de vida da consulta e do pagamento (complementa o diagrama ASCII acima)

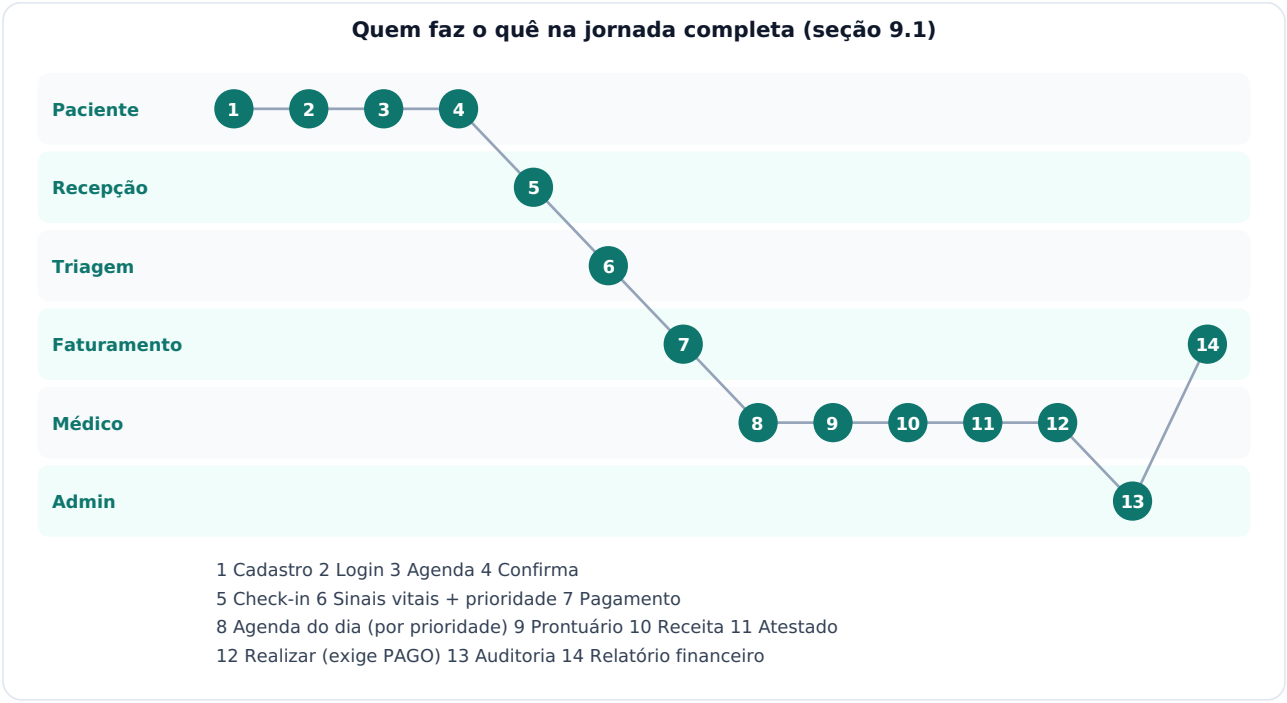


Figura 8 — Quem faz o quê em cada passo da jornada completa do paciente

PARTE 10 — CREDENCIAIS E DADOS DE ACESSO

10.1. Banco de Dados PostgreSQL

- **Host:** localhost (ou `db` dentro do Docker)
- **Porta:** 5432
- **Banco:** medflow
- **Usuário:** medflow
- **Senha:** medflow

Comando para acessar diretamente:

```
docker exec -it medflow-db psql -U medflow -d medflow
```

10.2. RabbitMQ

- **Painel Web:** http://localhost:15672
- **Usuário:** guest
- **Senha:** guest

10.3. Grafana

- **URL:** http://localhost:3000
- **Usuário:** admin
- **Senha:** admin

10.4. Prometheus

- **URL:** http://localhost:9090

10.5. Swagger (Documentação da API)

- **URL:** http://localhost:8080/swagger-ui.html

10.6. Frontend (Interface Web)

- **URL:** http://localhost:5173

10.7. Backend (API REST)

- **URL:** http://localhost:8080

10.8. Usuário Admin Padrão

- **Email:** admin@medflow.com
- **Senha:** admin

10.9. Chave Secreta JWT

Definida no arquivo `application.yml`:

```
jwt.secret: minha-chave-secreta-super-segura-para-jwt-do-medflow
```

Em produção, DEVE ser trocada por uma chave aleatória forte.

> ➕ **ADIÇÃO — Credenciais são apenas de desenvolvimento**
>
> Todos os usuários e senhas desta parte (`medflow/medflow` , `guest/guest` , `admin/admin` , `admin@medflow.com/admin`) e a chave JWT são **valores de demonstração**. Nunca devem ser usados em produção. Como este documento contém credenciais, **não o publique** em repositório ou canal aberto. Veja a Parte 13.

PARTE 11 — COMO RODAR O SISTEMA

11.1. Pré-requisitos

1. **Docker Desktop** instalado e rodando
2. **Git** instalado

11.2. Passo a passo

```
# 1. Clonar o repositório
git clone https://github.com/Samuka310/medflow.git

# 2. Entrar na pasta
cd medflow

# 3. Subir todos os containers
docker compose up -d

# 4. Aguardar ~60 segundos para tudo inicializar

# 5. Acessar
# Frontend: http://localhost:5173
# Backend/API: http://localhost:8080
# Swagger: http://localhost:8080/swagger-ui.html
# RabbitMQ: http://localhost:15672
# Grafana: http://localhost:3000
# Prometheus: http://localhost:9090
```

11.3. Parar o sistema

```
docker compose down
```

11.4. Parar e APAGAR todos os dados

```
docker compose down -v
```

O `-v` remove os volumes (dados do banco serão perdidos).

➕ ADIÇÃO — 11.5. Roteiro do primeiro uso (após subir o sistema)

Como só existe tela para login, cadastro, médicos e consultas, o preparo inicial é feito pelo **Swagger** (`http://localhost:8080/swagger-ui.html`), normalmente usando o botão *Authorize* com o token do login:

1. Faça login como administrador (`admin@medflow.com`) em `POST /auth/login` e copie o `token`.
2. Crie as especialidades: `POST /api/especialidades` (ADMIN).
3. Cadastre os médicos: `POST /api/medicos` (ADMIN), com CRM, valor da consulta e especialidades.
4. Crie os funcionários (recepção, triagem, faturamento): `POST /admin/funcionarios` (ADMIN).

5. Cadastre pacientes: pelo `Register` do site (`POST /auth/register`) ou pela recepção (`POST /api/pacientes`).
6. Agende a primeira consulta e siga o fluxo da seção 9.1.

➕ ADIÇÃO — 11.6. Mapa de portas e endereços

Componente	Container	Porta	Endereço
Frontend	`frontend`	5173	http://localhost:5173
Backend / API	`backend`	8080	http://localhost:8080
Swagger	`backend`	8080	http://localhost:8080/swagger-ui.html
PostgreSQL	`db`	5432	usar cliente SQL ou `psql`
RabbitMQ (painel)	`rabbitmq`	15672 (AMQP: 5672)	http://localhost:15672
Prometheus	`prometheus`	9090	http://localhost:9090
Grafana	`grafana`	3000	http://localhost:3000

➕ ADIÇÃO — 11.7. Comandos úteis

Comando	O que faz
`docker compose ps`	Mostra quais containers estão rodando e se estão saudáveis
`docker compose logs -f backend`	Acompanha, em tempo real, o que o backend está escrevendo
`docker compose restart backend`	Reinicia só o backend
`docker compose up -d --build`	Sobe tudo **reconstruindo** as imagens (necessário após mudar código)
`docker compose down`	Para tudo, mantendo os dados
`docker compose down -v`	Para tudo **e apaga os dados**
`curl http://localhost:8080/actuator/health`	Testa se o backend está saudável

➕ ADIÇÃO — 11.8. Problemas comuns

Sintoma	Causa provável	O que fazer
"Porta já em uso" (5432, 8080, 5173...)	Outro programa usa a porta	Feche o programa ou troque o número à **esquerda** do mapeamento (ex.: `5433:5432`)
Site abre, mas login falha logo após subir	Backend ainda inicializando	Aguarde ~60 s e veja `docker compose logs -f backend`
Backend não sobe	Banco ou RabbitMQ ainda não ficaram saudáveis	Veja `docker compose ps` e os logs do serviço com problema
Erro 401 ou 403 ao usar a API	Token expirado (24 h) ou role sem permissão	Faça login de novo e confira a tabela de permissões (Parte 5)
Quero começar do zero	Dados antigos no volume	`docker compose down -v` e depois `docker compose up -d`

PARTE 12 — GLOSSÁRIO DE TERMOS TÉCNICOS

Termo	Significado
API	Interface de Programação de Aplicações — conjunto de rotas que o frontend chama para se comunicar com o backend
REST	Estilo de arquitetura para APIs que usa verbos HTTP (GET, POST, PUT, DELETE)
JSON	Formato de dados usado na comunicação entre frontend e backend. Ex: `{"nome": "João"}`
JWT	Token de autenticação — uma string criptografada que prova quem é o usuário
UUID	Identificador Único Universal — um ID alfanumérico de 36 caracteres que nunca se repete
CRUD	Create, Read, Update, Delete — as 4 operações básicas de dados
ORM	Object-Relational Mapping — tecnologia que converte objetos Java em tabelas SQL
MVC	Model-View-Controller — padrão de organização de código
Container	Ambiente isolado e leve que contém tudo para rodar uma aplicação
Migration	Script SQL versionado que altera a estrutura do banco de dados
Endpoint	Um ponto de entrada da API. Ex: `/api/consultas` é um endpoint
Middleware	Código que intercepta requisições antes de chegarem ao destino final
BCrypt	Algoritmo de criptografia para senhas
CORS	Cross-Origin Resource Sharing — regra de segurança do navegador
Proxy Reverso	Servidor intermediário que redireciona requisições para outros servidores
Healthcheck	Verificação automática de saúde de um serviço
Scraping	Coleta automática de dados (neste caso, métricas do Prometheus)
SPA	Single Page Application — site que não recarrega ao navegar
Role	Papel/função de um usuário no sistema (ADMIN, MEDICO, etc.)
Bearer Token	Formato de envio do JWT: `Authorization: Bearer <token>`

➕ ADIÇÃO — 12.1. Glossário complementar

Termo	Significado
Frontend / Backend	Parte que o usuário vê / parte que processa e guarda os dados
Build	Compilar o código para gerar o programa executável
Deploy	Colocar o sistema no ar em um servidor
Framework / Biblioteca	Kit grande que dá a estrutura / conjunto de funções que você chama quando precisa
Dependência	Biblioteca externa da qual o projeto precisa

Termo	Significado
Anotação (`@`)	Marca no código que ativa um comportamento (`@Data` , `@PreAuthorize` , `@NotNull`)
Entity / DTO / Repository / Service / Controller	Camadas do backend (Parte 4): tabela, formato dos dados na API, acesso ao banco, regras, rotas
Header	Informação extra enviada junto com o pedido (ex.: `Authorization`)
Token	Código que prova a identidade do usuário sem reenviar a senha
Hash	Transformação de mão única (a senha vira um código que não volta ao original)
Stateless	O servidor não guarda "sessão"; cada pedido traz o token
Fila (queue)	Lista de mensagens esperando serem processadas
Produtor / Consumidor	Quem coloca mensagens na fila / quem as lê
Métrica / Dashboard	Número medido ao longo do tempo / painel de gráficos
Volume	Armazenamento permanente de um container
Actuator	Módulo do Spring Boot que expõe saúde e métricas
YAML	Formato de arquivo de configuração (`.yml`)
Profile (perfil)	Conjunto de configurações para um ambiente (dev, prod)
Ownership	Checagem de que o dado pertence a quem pede
CID	Código internacional de doença
Triagem	Avaliação inicial de sinais vitais e gravidade antes do médico
Prontuário	Registro clínico do atendimento
Estorno	Devolução/cancelamento de um pagamento já realizado
LGPD	Lei Geral de Proteção de Dados (Lei 13.709/2018)
XSS	Ataque que injeta script malicioso em uma página para roubar dados
LTS	Versão com suporte estendido

PARTE 13 — RECOMENDAÇÕES DE SEGURANÇA PARA PRODUÇÃO

Esta parte é uma adição. Ela lista o que deve ser revisto antes de colocar o MedFlow para atender pacientes reais.

13.1. Credenciais padrão que precisam ser trocadas

Item	Valor atual (dev)	Risco	Ação
Admin do sistema	`admin@medflow.com` / `admin`	Qualquer pessoa entra como administrador	Trocar a senha no primeiro acesso; usar senha forte
PostgreSQL	`medflow` / `medflow`	Acesso total aos dados de saúde	Senha forte, vinda de variável de ambiente
RabbitMQ	`guest` / `guest`	Leitura/escrita nas filas	Criar usuário próprio e desativar o `guest`
Grafana	`admin` / `admin`	Acesso às métricas do sistema	Trocar senha na primeira entrada
Chave JWT	`minha-chave-secreta-super-segura-para-jwt-do-medflow`	Quem a conhece **forja tokens** de qualquer usuário	Gerar chave aleatória de 32+ bytes (ex.: `openssl rand -base64 48`) e entregar por variável de ambiente

13.2. Reduzir o que fica exposto

- **Swagger e `/v3/api-docs`:** desativar ou proteger por login em produção (hoje são públicos).
- **`/actuator/prometheus`:** liberar só para a rede interna, não para a internet.
- **Painéis** do RabbitMQ (15672), Prometheus (9090) e Grafana (3000): não expor publicamente.
- **PostgreSQL (5432):** não publicar a porta para fora do servidor; só o backend precisa acessá-lo.

13.3. Token, sessão e navegador

- O token no `localStorage` pode ser roubado se houver uma falha de **XSS**. Uma alternativa mais segura é guardar o token em **cookie** `httpOnly`.
- O token vale **24 horas** e não há renovação. Considere token curto + *refresh token*.
- Limitar tentativas de login (proteção contra tentativa de senhas em massa).
- Restringir o **CORS** ao domínio real do frontend.

13.4. Comunicação e infraestrutura

- Usar **HTTPS (TLS)** em tudo, configurado no Nginx ou no Ingress/LoadBalancer.
- No Kubernetes, guardar senhas e chaves em **Secrets**, não em ConfigMap.
- Fazer **backups** do banco, por exemplo: `docker exec medflow-db pg_dump -U medflow medflow > backup.sql`, e testar a restauração.

13.5. Dados de saúde e LGPD

Prontuários, receitas e atestados são **dados pessoais sensíveis** (LGPD, art. 5º, II). Pontos de atenção:

- Registrar a **base legal** do tratamento e coletar apenas o necessário.
- A auditoria atual registra as **ações** listadas no documento; considere registrar também **quem leu** prontuários.
- Definir política de **retenção** do prontuário (há regras específicas para prontuário eletrônico) e de descarte.
- Criptografar backups e restringir quem acessa o banco.

Estas são orientações gerais; valide os requisitos legais com um especialista jurídico.

PARTE 14 — PONTOS DE ATENÇÃO ENCONTRADOS NA REVISÃO

Esta parte é uma adição. São inconsistências e lacunas percebidas ao ler o documento por inteiro, sem alterar o texto original. A revisão foi feita somente sobre o documento, sem acesso ao código-fonte; por isso cada item deve ser confirmado no projeto.

#	Onde	O que foi observado	Sugestão
1	4.1 (Consulta)	A lista e o diagrama de status **não incluem** `TRIADA`, mas o `TriagemService` (4.4) e o fluxo 9.1 usam esse status	Incluir `TRIADA` na lista e no diagrama (ver 9.2)
2	1.2 × 8.1	"5 caixas" na visão geral × 6 containers no Compose	Esclarecido na adição da Parte 1
3	2.1	Java 21 descrito como LTS mais recente; o Java 25 também é LTS	Ajustar o texto ou planejar migração
4	4.5 × 5.3	Swagger é público na 4.5, mas a tabela 5.3 o mostra liberado por role	Decidir a política e alinhar as duas seções (ver Parte 13)
5	3 × 4.5	A rota `/api/triagem` aparece como "via ConsultaController"; não há `TriagemController` na árvore de pastas	Confirmar no código e atualizar a árvore ou a tabela
6	4.1 × 4.2	`FuncionarioRequest` tem o campo `matricula`, mas a entidade `Funcionario` não lista esse campo	Confirmar se a coluna existe (migração V2) e documentar
7	4.4	`confirmar` não tem passo a passo, e `AdminService`, `MedicoService`, `EspecialidadeService`, `ExameService`, `ProntuarioService` também não	Documentar após conferir o código
8	4.1 e 4.2	`Exame` tem `status` sem valores definidos; `ExameDTO` e `EspecialidadeDTO` estão na árvore, mas não descritos na 4.2	Documentar campos e valores possíveis
9	4.4 (`realizar`)	A regra descrita só exige "não cancelada" e "pagamento PAGO": não exige triagem (`TRIADA`) nem prontuário	Decidir se isso é intencional e documentar
10	4.2	`PacienteDTO` contém `senha`	Garantir que nunca seja devolvida nas respostas
11	7.2	Só 5 telas; várias funções do backend não têm interface	Planejar as telas ou registrar a limitação
12	7.1	Versões de TypeScript, Vite e Axios aparecem como "-"	Preencher as versões
13	4.1	`status` e `role` são `String`, o que permite erro de digitação	Melhoria futura: usar `enum`
14	Capa e Parte 10	O documento contém senhas e a chave JWT	Guardar em local restrito (Parte 13)

PARTE 15 — AVALIAÇÃO: ATÉ ONDE UM INICIANTE CHEGA COM ESTE DOCUMENTO

Esta parte é uma adição e traz uma opinião sobre o documento: o que uma pessoa sem nenhum conhecimento prévio em programação consegue aprender lendo-o.

15.1. Veredito

O documento é **excelente como mapa e como referência**: explica o que cada peça é e para que serve, traz analogias acessíveis, cobre todas as rotas, regras e fluxos, e mostra como rodar tudo. Lido do começo ao fim por alguém que **nunca teve contato com programação**, ele leva de forma confortável até o **nível 2 (modelo mental)**, e chega de forma **parcial** ao nível 3 (operador): a pessoa entende o que fazer, mas sem prática ainda fica insegura. Isso é **compreensão**, não **habilidade**: ler não substitui fazer.

Com as adições desta versão (exemplos, figuras e exercícios), o ponto de chegada sobe para o **nível 3 completo** desde que a pessoa **faça os exercícios**. Os níveis 4 e 5 exigem estudar fundamentos de programação e praticar, o que nenhum documento de referência entrega sozinho.

15.2. A escala usada

Nível	Nome	O que a pessoa consegue fazer
0	Sem contato	Não sabe do que o sistema se trata
1	Vocabulário	Explica o que são frontend, backend, API, banco, token, container, fila
2	Modelo mental	Descreve o caminho de uma ação (ex.: agendar) e diz por que cada peça existe
3	Operador	Sobe o sistema, faz login, testa a API no Swagger, lê logs, consulta o banco, percorre o fluxo completo
4	Leitor de código	Navega no projeto, entende um Controller ou Service e faz pequenas mudanças guiadas
5	Desenvolvedor autônomo	Cria funcionalidades novas, depura problemas e escreve testes



Figura 9 — Nível alcançado por cada caminho de estudo, partindo de zero

15.3. O que funciona muito bem para um iniciante

Ponto forte	Por que ajuda
Padrão "O que é? / Pra que serve aqui?" repetido em cada tecnologia	Dá a mesma estrutura mental para tudo o que é novo
Analogias (restaurante, crachá, bibliotecário, estufa)	Transformam conceitos abstratos em algo do dia a dia
Mapa completo de rotas e permissões	A pessoa vê exatamente quem pode fazer o quê
Fluxo ponta a ponta (Parte 9)	Junta todas as peças numa história com começo e fim
Glossário e credenciais de acesso	Permitem consultar e testar sem ficar perdido

15.4. O que ainda atrapalha quem parte do zero

Dificuldade	Detalhe	Como esta versão ajuda
Muito vocabulário novo de uma vez	A Parte 2 apresenta dezenas de ferramentas em poucas páginas	Mapa de tecnologias por camada, glossário complementar e ordem de leitura sugerida (15.5)
Nenhum exemplo concreto de dados	O original descreve campos, mas não mostra uma requisição real	Exemplos de JSON, `curl`, SQL e código ilustrativo
Sem exercícios	Ler passivamente não cria habilidade	Exercícios práticos (15.6)
Pressupõe fundamentos	Terminal, HTTP/JSON, SQL, lógica de programação, Java básico e Git não são explicados	Lista de pré-requisitos (15.7)

Dificuldade	Detalhe	Como esta versão ajuda
Só 5 telas no frontend	Boa parte do sistema só é testável pelo Swagger	Roteiro do primeiro uso (11.5)
Informação só em texto	Difícil enxergar relações	9 figuras

15.5. Ordem de leitura sugerida para iniciantes

1. **Parte 1** (visão geral) e **Parte 12** (glossário): dão o vocabulário e a ideia geral.
2. **Parte 9** (o fluxo completo): a "história" do sistema, agora com os diagramas.
3. **Parte 11** (como rodar) e os **exercícios 1 a 4**: hora de ver o sistema funcionando.
4. **Parte 5** (segurança) e a **seção 4.5** (rotas): o que cada perfil pode fazer.
5. **Parte 4** (camadas do backend) e **Parte 3** (pastas): como o código é organizado.
6. **Partes 2, 6, 7 e 8**: use como **consulta**, conforme as dúvidas aparecerem. Não precisa decorar a Parte 2.

15.6. Exercícios práticos (com o resultado esperado)

#	Nível	Exercício	Resultado esperado
1	3	Siga a Parte 11, rode `docker compose up -d` e depois `docker compose ps`	6 containers ativos (db, rabbitmq, backend, frontend, prometheus, grafana)
2	3	Execute `curl http://localhost:8080/actuator/health`	Resposta indicando que a aplicação está **UP**
3	3	No Swagger, faça login como admin, copie o token e use o botão *Authorize*	Rotas protegidas passam a responder sem 401
4	3	Siga o roteiro 11.5: crie especialidade, médico, funcionário e paciente	Dados aparecem nas listagens (`GET /api/medicos` , `/api/pacientes`)
5	3	Agende uma consulta e consulte o pagamento pelo `pagamentoId` devolvido	Existe um pagamento com status **PENDENTE**, criado automaticamente
6	3	Tente agendar uma segunda consulta 30 min depois, com o mesmo médico	Recusada por conflito de horário
7	3	Tente `realizar` a consulta sem ter pago	Recusada; depois de pagar, funciona
8	3	Percorra o fluxo 9.1 inteiro e depois chame `GET /api/auditoria`	Uma entrada de auditoria por ação importante realizada
9	3	Abra `psql` e rode `SELECT status, COUNT(*) FROM consulta GROUP BY status;`	Contagem de consultas por status
10	3	Abra o Prometheus (porta 9090) e pesquise `http_server_requests_seconds_count`	Série com o número de requisições do backend
11	4	Em `ConsultaService.java`, ache as regras de 30 min e 2 horas e localize o teste correspondente em `ConsultaServiceTest.java`	Você consegue apontar a linha da regra e o teste que a cobre

#	Nível	Exercício	Resultado esperado
12	4	Mude o cancelamento de 2 horas para 1 hora e rode `mvn test`	Algum teste falha; entender **por quê** é o objetivo

15.7. O que estudar para subir do nível 3 para o 4 e 5

O documento assume que a pessoa já sabe (ou vai aprender em outro lugar):

- **Lógica de programação:** variáveis, condicionais (`if`), repetições, funções.
- **Java básico:** classes, objetos, métodos, interfaces, anotações (`@`).
- **HTTP e JSON:** o que são requisição, resposta, headers, verbos e códigos de status.
- **SQL básico:** `SELECT`, `WHERE`, `JOIN`, `INSERT`, `UPDATE`.
- **Terminal:** navegar entre pastas e executar comandos.
- **Git básico:** clonar, criar commits, ramificações.

Minha estimativa pessoal, que varia muito de pessoa para pessoa: com dedicação regular, alguns meses de estudo dessas bases somados à prática guiada neste projeto levam ao nível 4; o nível 5 vem de construir funcionalidades reais ao longo de bastante tempo.

15.8. Conclusão

Para quem nunca programou, este documento é um **ótimo primeiro mapa**, mas **não é um curso**. Ele ensina **o que existe, para que serve e como as partes se encaixam**. Para aprender a **fazer**, o caminho é combinar a leitura com os exercícios acima, estudar os fundamentos da seção 15.7 e mexer no projeto aos poucos.

> **Fim do Documento**
>
> Este documento cobre 100% da estrutura atual do MedFlow versão 2.9. Qualquer adição futura (Fases 2.10, 2.11, 2.12) será adicionada como apêndice.